

DOSSIER DE PROJET

Titre Professionnel

Concepteur Développeur d'Applications — Niveau 6 RNCP

AnimeTracker

Application mobile de suivi et gestion d'animés

CUENOT ANTONIN

Mai 2026

Sommaire

PARTIE 1 — PRÉSENTATION DU PROJET

- 1.1 Présentation du candidat et contexte de formation
- 1.2 Présentation générale du projet
- 1.3 Objectifs et périmètre fonctionnel
- 1.4 Stack technique — vue d'ensemble

PARTIE 2 — ANALYSE DES BESOINS

- 2.1 Acteurs et rôles
- 2.2 User Stories par fonctionnalité
- 2.3 Règles de gestion
- 2.4 Contraintes techniques et non-fonctionnelles

PARTIE 3 — CONCEPTION

- 3.1 Architecture générale
- 3.2 Modèle Conceptuel de Données (MCD)
- 3.3 Modèle Logique de Données (MLD)
- 3.4 Modèle de classes
- 3.5 Diagrammes UML (cas d'utilisation, séquences)
- 3.6 Maquettes et wireframes
- 3.7 Charte graphique

PARTIE 4 — RÉALISATION TECHNIQUE

- 4.1 Justification des choix technologiques
- 4.2 Architecture backend
- 4.3 Architecture frontend
- 4.4 Base de données
- 4.5 Fonctionnalités clés — extraits de code commentés

4.6 Infrastructure Docker

PARTIE 5 — TESTS ET QUALITÉ

5.1 Stratégie de tests

5.2 Tests automatisés Jest — résultats

5.3 Tests manuels sur émulateur Android

5.4 Couverture fonctionnelle

PARTIE 6 — BILAN ET RÉTROSPECTIVE

6.1 Ce qui a bien fonctionné

6.2 Difficultés rencontrées et solutions apportées

6.3 Axes d'amélioration

6.4 Compétences acquises

ANNEXES

A Liste complète des endpoints API

B Schéma de base de données complet

C Captures d'écran de l'application

D Résultats Jest complets

Partie 1 — Présentation du projet

1.1 Présentation du candidat

Je m'appelle Antonin Cuenot. Ce dossier est réalisé dans le cadre de la validation du titre professionnel Concepteur Développeur d'Applications (CDA), enregistré au RNCP de niveau 6.

J'enchaîne ce titre directement après l'obtention du titre professionnel Développeur Web et Web Mobile (DWWM, niveau 5 RNCP), validé en 2024. Le CDA représente la suite naturelle de cette progression : là où le DWWM couvrait le développement web, le CDA monte en exigence sur la conception logicielle, l'architecture des applications et la rigueur de la modélisation.

Le projet présenté ici, AnimeTracker, est une application mobile développée de bout en bout : de la base de données jusqu'à l'interface utilisateur sur Android. Lors du DWWM, j'avais présenté un site web — pour ce CDA, j'avais comme objectif de travailler sur quelque chose de différent et d'explorer le développement mobile, une compétence que je n'avais pas encore eu l'occasion de pratiquer. Le choix du thème des animés n'est pas anodin non plus : c'est un univers que je connais bien, ce qui m'a permis de me concentrer sur les défis techniques sans avoir à chercher les règles métier.

1.2 Présentation du projet

AnimeTracker est une application mobile destinée aux fans d'animés. Elle permet de gérer sa bibliothèque personnelle de séries, d'échanger des avis, de discuter dans des groupes par animé, et de suivre l'activité d'autres utilisateurs.

L'idée est simple : au lieu de jongler entre plusieurs sites pour noter ses animés, voir les avis des autres ou chercher des gens avec les mêmes goûts, tout se passe dans une seule application. Les données des animés (titres, synopsis, affiches, notes) viennent de l'API publique Jikan, qui agrège les informations de MyAnimeList.

1.3 Objectifs et périmètre fonctionnel

L'application couvre les fonctionnalités suivantes :

- Authentification sécurisée (inscription, connexion, déconnexion avec invalidation de token)
- Bibliothèque personnelle — ajouter un animé, suivre son avancement épisode par épisode, définir un statut (en cours, terminé, abandonné...), marquer en favori
- Avis — rédiger une critique publique ou privée, noter de 1 à 10, liker les avis des autres
- Messagerie privée — conversations directes entre utilisateurs
- Groupes de discussion — un groupe par animé, rejoint librement
- Social — s'abonner à d'autres utilisateurs, consulter leurs profils et bibliothèques
- Profil utilisateur — modifier son pseudo, sa bio, son mot de passe, voir ses statistiques

Ce qui est volontairement hors périmètre : la modération de contenu, les notifications push en temps réel, l'upload d'avatar personnalisé, et la version iOS (développée uniquement sur émulateur Android pour ce projet).

1.4 Stack technique — vue d'ensemble

L'application est construite sur une architecture client-serveur classique, avec une séparation nette entre l'application mobile et l'API backend.

Frontend mobile

- - React Native + Expo — application multiplateforme (Android/iOS)
 - TypeScript — typage statique sur l'ensemble du frontend
 - Redux Toolkit — gestion de l'état global de l'application
 - React Navigation — navigation entre écrans (Stack + Bottom Tabs)
 - Axios — client HTTP avec gestion automatique des tokens JWT

Backend API REST

- - Node.js + Express — serveur HTTP léger et rapide
 - TypeScript — typage statique côté serveur
 - Sequelize — ORM pour interagir avec PostgreSQL
 - Zod — validation des données entrantes
 - JSON Web Token (JWT) — authentification sans état

Infrastructure

- - PostgreSQL — base de données relationnelle principale
 - Redis — cache pour la blacklist des tokens invalidés
 - Docker + docker-compose — environnement de développement reproductible

Veille technologique

Avant de démarrer le développement d'AnimeTracker, j'ai effectué une veille technologique pour identifier les outils et frameworks les mieux adaptés au projet. Cette veille couvre les choix clés : le framework mobile, le backend, la base de données, l'authentification, la gestion d'état et l'ORM.

1. Développement mobile — React Native vs les alternatives

L'objectif étant de produire une application mobile Android (avec portabilité iOS possible), plusieurs approches ont été comparées.

Framework	Langage	Avantages	Inconvénients
React Native	JavaScript / TypeScript	Code unique Android + iOS, large communauté, écosystème npm, composants natifs réels	Performances légèrement inférieures au 100% natif, modules natifs parfois complexes
Flutter	Dart	Performances excellentes, rendu cohérent, widgets personnalisés	Langage Dart peu répandu, taille des APK plus lourde, écosystème plus jeune
Ionic / Capacitor	HTML/CSS/JS	Réutilise les compétences web, rapide à démarrer	Rendu WebView (pas natif), performances limitées, look moins natif
Développement natif	Kotlin (Android) / Swift (iOS)	Performances maximales, accès total aux APIs	Deux bases de code distinctes à maintenir, temps de développement doublé
Progressive Web App	HTML/CSS/JS	Pas d'installation requise, simple à déployer	Accès limité aux APIs natives, pas distribué via les stores

Choix retenu : React Native + Expo

React Native s'impose comme le choix le plus cohérent : une seule base de code TypeScript pour Android et iOS, un écosystème mature (Meta, Microsoft, Shopify l'utilisent en production), et surtout une continuité directe avec mes compétences JavaScript acquises pendant le DWWWM. Expo ajoute un outillage qui accélère considérablement le démarrage sans sacrifier la flexibilité.

2. Backend — Node.js/Express vs les alternatives

Framework	Langage	Avantages	Inconvénients
Node.js + Express	JavaScript / TypeScript	Même langage que le frontend, event-loop non bloquant, écosystème npm immense, minimaliste	Peu structuré (liberté = responsabilité d'organisation)
NestJS	TypeScript	Architecture très structurée (modules, décorateurs), inspiré d'Angular	Verbeux, complexe pour un projet de taille moyenne, courbe d'apprentissage élevée
Django / FastAPI	Python	Rapide à développer, Django très complet out-of-the-box	Langage différent du frontend, perd la cohérence full-JS
Spring Boot	Java	Robuste, très utilisé en entreprise, typage fort	Verbeux, démarrage lent, lourd pour une API REST simple
Laravel	PHP	Complet, bonnes conventions, écosystème riche	PHP peu valorisé en mobile, langage différent du frontend

Choix retenu : Node.js + Express + TypeScript

Le principal avantage de Node.js est la cohérence full-stack : TypeScript des deux côtés, interfaces partagées, une seule compétence de langage couvre le projet entier. Express a été préféré à NestJS car sa simplicité correspond à la taille du projet — NestJS aurait introduit de la complexité architecturale sans valeur ajoutée à ce stade.

3. Base de données — PostgreSQL vs les alternatives

SGBD	Type	Avantages	Inconvénients
PostgreSQL	Relationnel (SQL)	ACID, relations complexes, UUID natif, types ENUM, performances excellentes, open source	Schéma rigide (mais c'est un avantage ici)
MySQL / MariaDB	Relationnel (SQL)	Très répandu, performances solides, grande communauté	Moins de types natifs avancés, ENUM moins flexible
MongoDB	NoSQL (documents)	Flexibilité du schéma, facile à démarrer, JSON natif	Jointures complexes coûteuses, cohérence ACID limitée, mauvais choix si les données sont relationnelles
SQLite	Relationnel embarqué	Zéro configuration, idéal pour mobile en local	Non adapté à un backend multi-utilisateurs
Supabase	PostgreSQL managé	BaaS complet, auth intégrée, réeltime	Dépendance à un service tiers, moins de contrôle

Choix retenu : PostgreSQL

Le modèle de données d'AnimeTracker est fortement relationnel — utilisateurs, animés, avis, abonnements, messages : tout est lié par des clés étrangères et des contraintes d'intégrité. MongoDB aurait été un mauvais choix ici car la flexibilité des documents ne compense pas la perte des jointures et des contraintes. PostgreSQL est aussi le standard recommandé pour les projets CDA, et son support natif des UUID et des types ENUM renforce l'intégrité des données directement au niveau base.

4. Authentification — JWT vs les alternatives

Mécanisme	Principe	Avantages	Inconvénients
JWT (JSON Web Token)	Token signé, stocké côté client	Stateless, scalable, idéal pour mobile, vérification sans BDD	Révocation complexe (nécessite une blacklist)
Sessions serveur	ID de session côté serveur (Redis/BDD)	Révocation immédiate simple, contrôle total	Stateful, nécessite un stockage partagé entre serveurs
OAuth 2.0 (Google, GitHub...)	Délégation à un fournisseur tiers	UX simplifiée (pas de mot de passe), sécurité déléguée	Dépendance à un service externe, implémentation complexe
Passport.js	Middleware d'authentification Node	Compatible JWT + OAuth, nombreuses stratégies disponibles	Surcouche supplémentaire, complexité accrue

Choix retenu : JWT + blacklist Redis

JWT est la solution la plus adaptée aux applications mobiles : le token est stocké localement (SecureStore d'Expo), chaque requête est auto-suffisante sans aller-retour en base pour vérifier la session. Le seul inconvénient de JWT — la difficulté à révoquer un token — est résolu par la blacklist Redis : à la déconnexion, le token est invalide immédiatement et reste en mémoire Redis jusqu'à son expiration naturelle.

5. Gestion d'état frontend — Redux vs les alternatives

Solution	Avantages	Inconvénients	Adapté si...
Redux Toolkit	Standard de l'industrie, DevTools puissants, optimisation fine des re-renders, AsyncThunk intégré	Boilerplate plus important, courbe d'apprentissage	Application complexe avec beaucoup d'états partagés
Zustand	Minimaliste, API simple, peu de boilerplate	Moins structuré, DevTools limités	Petite application, état simple
MobX	Réactivité automatique, moins de code	Magie implicite, debug parfois difficile	Préférence pour le style orienté-objet
Context API (React)	Natif React, aucune dépendance	Re-renders en cascade, pas optimisé pour gros état	État simple, peu de composants concernés
React Query / TanStack	Gestion cache serveur excellente	Ne gère pas l'état UI local, dépendance supplémentaire	Applications data-fetching intensif

Choix retenu : Redux Toolkit

AnimeTracker gère plusieurs domaines d'état interdépendants (auth, bibliothèque, avis, social, messagerie, groupes). Redux Toolkit est la solution la plus adaptée à cette complexité : les slices organisent l'état par domaine, les AsyncThunk standardisent les appels API, et Redux DevTools permettent d'inspecter chaque action en développement. Zustand aurait été insuffisant pour gérer les interactions entre slices.

6. ORM — Sequelize vs les alternatives

ORM	Avantages	Inconvénients
Sequelize	Mature, supporte PostgreSQL/MySQL/SQLite, migrations, associations, bonne documentation	API parfois verbeuse, typage TypeScript à configurer manuellement
Prisma	DX excellente, schéma déclaratif, typage TypeScript automatique et précis	Génération de client nécessaire, moins flexible sur les requêtes complexes
TypeORM	Decorateurs TypeScript, familier si Angular, migrations	Bugs connus, maintenance ralentie, moins stable que Sequelize
Knex.js (query builder)	Contrôle SQL total, performances maximales	Pas d'abstraction objet, plus de code SQL à écrire manuellement
SQL brut (pg)	Performances maximales, contrôle absolu	Pas de protection injection automatique, verbeux, risqué

Choix retenu : Sequelize

Sequelize est l'ORM le plus établi de l'écosystème Node.js. Malgré une syntaxe parfois plus verbale que Prisma, il offre une maturité et une stabilité éprouvées. Son système d'associations (hasMany, belongsTo, belongsToMany) mappe parfaitement le modèle relationnel d'AnimeTracker. Les requêtes paramétrées automatiques protègent contre les injections SQL sans effort supplémentaire.

7. Infrastructure — Docker vs les alternatives

Approche	Avantages	Inconvénients
Docker + docker-compose	Environnement reproductible, isolation des services, version des dépendances fixée, proche de la production	Overhead mémoire, complexité initiale, problème inotify sur Windows
Installation locale native	Simple, performances natives, pas de surcharge VM	"Ça marche sur ma machine" — non reproductible entre développeurs
Services cloud (Supabase, Railway)	Zéro configuration serveur, scalabilité automatique	Dépendance externe, coûts potentiels, moins de contrôle
Machines virtuelles	Isolation complète, proche production	Lentes à démarrer, consommatrices de ressources

Choix retenu : Docker + docker-compose

Docker garantit que l'environnement est identique quelle que soit la machine. PostgreSQL 15-alpine et Redis 7-alpine dans des conteneurs versionnés évitent les conflits avec des installations locales. Le docker-compose.yml devient à lui seul la documentation de l'infrastructure — un développeur qui rejoint le projet n'a besoin que de Docker et d'un fichier .env pour démarrer.

Synthèse des choix technologiques

Domaine	Technologie retenue	Principale raison du choix
Application mobile	React Native + Expo	Code unique Android/iOS, cohérence TypeScript avec le backend
Backend API	Node.js + Express	Même langage que le frontend, event-loop adapté aux APIs REST
Base de données	PostgreSQL	Données relationnelles, contraintes d'intégrité, UUID natifs
Cache / blacklist	Redis	Vérification JWT ultra-rapide en mémoire
Authentification	JWT + blacklist Redis	Stateless, idéal pour mobile, révocation possible
Gestion d'état	Redux Toolkit	Complexité multi-domaine, DevTools, AsyncThunk
ORM	Sequelize	Maturité, associations, protection injection SQL
Infrastructure	Docker + docker-compose	Reproductibilité, isolation des services
Typage	TypeScript (full-stack)	Détection d'erreurs à la compilation, documentation vivante

Partie 2 — Analyse des besoins

Avant d'écrire la moindre ligne de code, il était nécessaire de bien définir ce que l'application devait faire — et pour qui. Cette partie décrit les acteurs impliqués, les fonctionnalités attendues sous forme de user stories, les règles qui régissent les données, et les contraintes à respecter.

2.1 Acteurs et rôles

L'application distingue plusieurs types d'utilisateurs, chacun avec des droits et des usages différents.

Acteur	Description	Accès
Visiteur	Utilisateur non connecté	Consultation du catalogue uniquement
Utilisateur	Compte créé et connecté	Toutes les fonctionnalités de l'application
Modérateur	Utilisateur avec droits élargis	Gestion des groupes de discussion
Administrateur	Gestion complète	Modération, suspension de comptes, signalements
Jikan API	Système externe (MyAnimeList)	Fournisseur des données animés (lecture seule)

Le rôle ADMIN dispose d'un panneau d'administration complet (stats globales, gestion des utilisateurs, modération des avis et des messages de groupe). L'accès est sécurisé par le middleware `requireRole('ADMIN')` — toute tentative d'accès par un non-admin retourne une erreur 403.

2.2 User Stories par fonctionnalité

Les user stories sont organisées par domaine fonctionnel. Chaque story suit le format : "En tant que [acteur], je veux [action] afin de [objectif]". La colonne Priorité indique si la fonctionnalité fait partie du MVP (Haute), de la v1.5 (Moyenne) ou d'une version future (Basse).

A. Authentification et profil

ID	Priorité	En tant que	Je veux	Critères clés
US01	Haute	Visiteur	Créer un compte	Email unique, mot de passe min. 8 caractères
US02	Haute	Utilisateur	Me connecter	JWT, session persistante, message d'erreur clair
US03	Haute	Utilisateur	Me déconnecter	Token invalidé immédiatement (blacklist Redis)
US04	Moyenne	Utilisateur	Modifier mon profil	Pseudo unique, bio max 500 caractères
US06	Moyenne	Utilisateur	Voir le profil d'un autre	Pseudo, bio, stats, avis publics uniquement

B. Bibliothèque personnelle

ID	Priorité	En tant que	Je veux	Critères clés
US11	Haute	Utilisateur	Ajouter un animé à ma liste	Ajout en 1 clic, statut "À voir" par défaut
US12	Haute	Utilisateur	Marquer "En cours"	Activation du suivi épisode par épisode
US13	Haute	Utilisateur	Marquer "Terminé"	Date de fin enregistrée, historique conservé
US14	Moyenne	Utilisateur	Marquer "Abandonné"	Possibilité de reprendre plus tard
US15	Haute	Utilisateur	Suivre ma progression	Compteur épisodes vus / total, mis à jour manuellement
US16	Moyenne	Utilisateur	Voir mes statistiques	Temps total, nombre d'animés, note moyenne

C. Avis et évaluations

ID	Priorité	En tant que	Je veux	Critères clés
US17	Haute	Utilisateur	Noter un animé (sur 10)	Note de 1 à 10, modifiable
US18	Haute	Utilisateur	Écrire un avis	Texte libre, max 2000 caractères
US19	Haute	Utilisateur	Choisir la visibilité	Public ou privé, privé par défaut
US20	Moyenne	Utilisateur	Modifier ou supprimer mon avis	Édition libre, suppression avec confirmation
US21	Moyenne	Utilisateur	Liker un avis public	1 like max par utilisateur, compteur visible

D. Messagerie privée

ID	Priorité	En tant que	Je veux	Critères clés
US25	Moyenne	Utilisateur	Envoyer un message privé	Texte uniquement, horodatage
US26	Moyenne	Utilisateur	Lire mes conversations	Liste des conversations, historique complet
US27	Basse	Utilisateur	Supprimer une conversation	Suppression côté utilisateur, confirmation obligatoire

E. Groupes de discussion

ID	Priorité	En tant que	Je veux	Critères clés
US28	Moyenne	Utilisateur	Rejoindre le groupe d'un animé	1 groupe officiel par animé, rejoint en 1 clic
US32	Moyenne	Utilisateur	Publier un message dans le groupe	Max 1000 caractères, horodatage, auteur visible
US33	Moyenne	Utilisateur	Lire les messages du groupe	Affichage chronologique, scroll infini
US34	Basse	Utilisateur	Quitter un groupe	Possibilité de rejoindre à nouveau

F. Social — abonnements

ID	Priorité	En tant que	Je veux	Critères clés
US22	Moyenne	Utilisateur	Suivre un autre utilisateur	Bouton "S'abonner" sur son profil
US23	Moyenne	Utilisateur	Ne plus suivre un utilisateur	Bouton "Se désabonner", pas de notification
US24	Basse	Utilisateur	Voir mes abonnés / abonnements	Deux listes séparées, liens vers profils

2.3 Règles de gestion

Ces règles définissent les contraintes métier appliquées dans l'ensemble de l'application. Elles ont guidé les choix d'implémentation côté backend.

- Les avis sont privés par défaut — un utilisateur doit explicitement choisir de rendre son avis public. Cela évite la publication accidentelle de données personnelles.
- Un seul avis par utilisateur et par animé — si l'utilisateur rédige un nouvel avis, il remplace le précédent (upsert).
- Un seul groupe officiel par animé — créé automatiquement à la première demande d'adhésion. Il ne peut pas être supprimé.
- Les données des animés (titre, synopsis, affiches, note) ne sont jamais stockées localement de façon permanente — elles sont toujours récupérées depuis l'API Jikan au moment de l'affichage.
- Un utilisateur ne peut pas se suivre lui-même — cette action retourne une erreur 400.
- Un token JWT invalidé (déconnexion) est placé en blacklist Redis jusqu'à sa date d'expiration naturelle — cela garantit qu'un token volé après déconnexion ne peut plus être utilisé.
- Les mots de passe sont hachés avec bcrypt (coût 10) avant tout stockage — le mot de passe en clair n'est jamais conservé.
- Toutes les routes de l'API (sauf /auth/register et /auth/login) nécessitent un token JWT valide dans l'en-tête Authorization.

2.4 Contraintes techniques et non-fonctionnelles

Sécurité

- Rate limiting sur les routes d'authentification (max 20 requêtes / 15 minutes par IP) — protection contre les attaques par force brute.
- Validation stricte de toutes les données entrantes avec Zod — aucune donnée non validée ne passe dans la base.

- En-têtes de sécurité HTTP gérés par Helmet (Express).
- CORS configuré pour n'accepter que les origines autorisées.

Performance

- Redis utilisé comme cache pour la blacklist JWT — la vérification d'un token ne génère pas de requête SQL.
- Les appels à l'API Jikan sont faits directement depuis le client mobile — le backend n'est pas un proxy inutile pour ces données.

Maintenabilité

- TypeScript sur l'ensemble du projet (frontend et backend) — les erreurs de type sont détectées à la compilation, pas en production.
- Pattern Controllers / Services côté backend — la logique métier est isolée des routes HTTP, ce qui facilite les tests unitaires.
- Tous les modèles Sequelize sont mockés dans les tests — aucune connexion réelle à la base de données n'est nécessaire pour faire tourner la suite Jest.

Environnement de développement

- L'ensemble de l'infrastructure (PostgreSQL + Redis + backend) est conteneurisé avec Docker — un seul "docker compose up" suffit à démarrer l'environnement complet.
- L'émulateur Android utilise l'adresse 10.0.2.2 pour accéder au backend sur la machine hôte (équivalent de localhost depuis l'émulateur).

Partie 3 — Conception

Cette partie couvre les choix d'architecture et les modèles produits avant et pendant le développement : organisation des couches, structure de la base de données, modèle de classes, diagrammes UML, maquettes et charte graphique.

3.1 Architecture générale

AnimeTracker repose sur une architecture client-serveur en trois couches distinctes : le client mobile, l'API backend et la couche données. Cette séparation permet de faire évoluer chaque couche indépendamment.

Couche	Technologie	Rôle
Client mobile	React Native + Expo	Interface utilisateur, navigation, appels API
API REST	Node.js + Express	Logique métier, authentification, validation
Données	PostgreSQL + Redis	Persistance relationnelle + cache JWT

Organisation du backend — pattern en trois niveaux :

- Routes (Express) — reçoivent les requêtes HTTP, appliquent les middlewares, délèguent au controller
- Controllers — extraient les paramètres de la requête, appellent le service, formatent la réponse
- Services — contiennent toute la logique métier, accèdent à la base via les modèles Sequelize

Ce découpage garantit que la logique métier est indépendante du transport HTTP, ce qui facilite les tests unitaires : les services sont testables sans démarrer le serveur.

Flux d'une requête authentifiée :

Client mobile

- POST /auth/login {email, password}
- Middleware Zod : valide le corps de la requête
- Middleware authenticate : vérifie le JWT (sauf /auth/*)
- Controller : extrait les paramètres
- Service : hash, compare, génère le token
- Réponse : { success: true, data: { token, user } }

3.2 Modèle Conceptuel de Données (MCD)

Le MCD identifie les entités métier et leurs relations avant de penser à la structure SQL. AnimeTracker repose sur 9 entités principales.

Entité	Description	Relations principales
User	Compte utilisateur	Possède des UserAnime, des Review, des Follow, des Message
Anime	Cache des données Jikan	Suivi par des UserAnime, évalué par des Review, lié à un Group
UserAnime	Animé dans la bibliothèque	Lie User et Anime avec statut et progression
Review	Avis sur un animé	Lie User et Anime, peut être liké via ReviewLike
ReviewLike	Like d'un avis	Lie User et Review (contrainte d'unicité)
Follow	Abonnement entre users	Lie User (suiveur) et User (suivi)
Conversation	Messagerie privée	Contient des Message, a des ConversationParticipant
Group	Groupe de discussion	Lié à un Anime, contient des GroupMember et GroupMessage
Message / GroupMessage	Messages des conversations/groupes	Liés à leur conteneur respectif

Relations clés :

- User ↔ Anime : relation N:N via UserAnime (un utilisateur peut avoir plusieurs animés, un animé peut être dans plusieurs bibliothèques)
- User ↔ User : relation N:N réflexive via Follow (un utilisateur peut suivre plusieurs autres utilisateurs)
- User ↔ Review : relation 1:N avec contrainte d'unicité (un seul avis par utilisateur par animé)
- Anime ↔ Group : relation 1:1 pour les groupes officiels (un groupe officiel maximum par animé)

3.3 Modèle Logique de Données (MLD)

Le MLD traduit le MCD en tables SQL concrètes avec leurs types, clés primaires (UUID sauf pour les animés) et contraintes de clés étrangères.

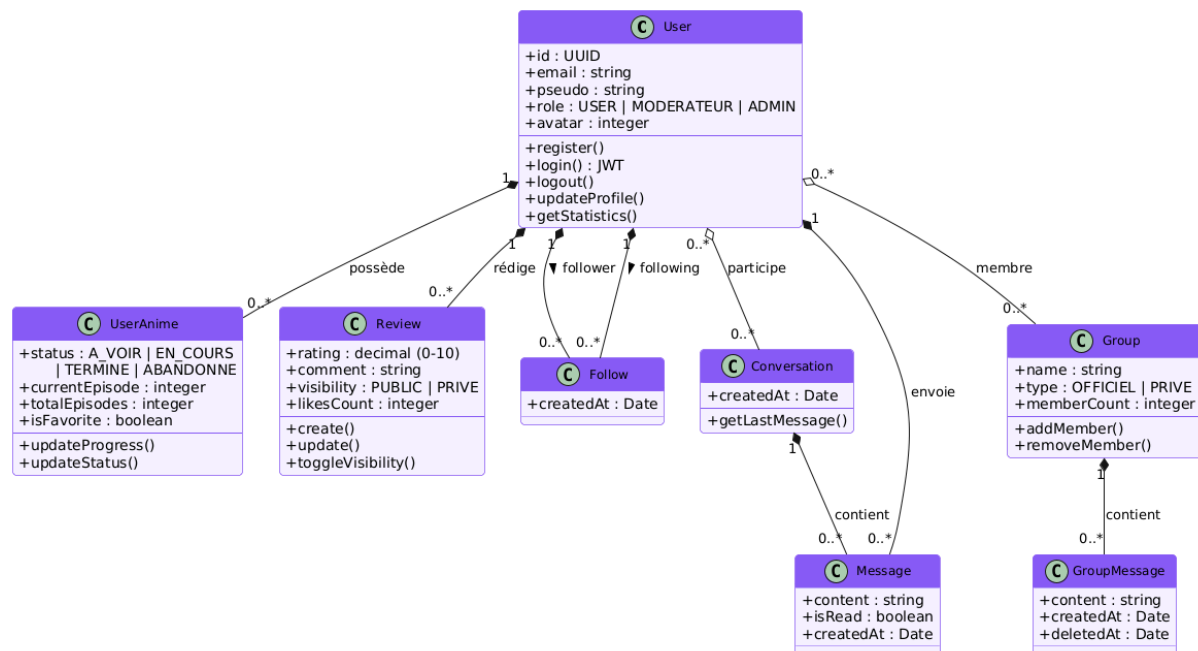
Pourquoi des UUID et non des entiers auto-incrémentés ?

Les UUID (Universally Unique Identifiers) présentent deux avantages pour ce projet : ils ne révèlent pas le volume de données (un id=42 indique qu'il y a au moins 42 utilisateurs en base) et ils permettent de générer des identifiants côté client sans risque de collision. Les animés utilisent toutefois l'entier fourni par Jikan/MyAnimeList, car c'est le référentiel partagé.

Table	Clé primaire	Colonnes principales	Clés étrangères
users	id_user (UUID)	email, password, pseudo, bio, role, avatar, is_active	—
user_anime	id_user_anime (UUID)	status, episodes_watched, is_favorite, started_at, completed_at	id_user → users, id_anime → animes
animes	id_anime (Integer)	title, synopsis, image_url, episodes, score, status	—
reviews	id_review (UUID)	rating, comment, visibility, likes_count	id_user → users, id_anime → animes
review_likes	id_like (UUID)	created_at	id_review → reviews, id_user → users
follows	id_follow (UUID)	created_at	id_follower → users, id_following → users
conversations	id_conversation (UUID)	created_at, updated_at	—
conversation_participants	id_participant (UUID)	joined_at	id_conversation → conversations, id_user → users
messages	id_message (UUID)	content, is_read, created_at	id_conversation → conversations, id_sender → users
groups	id_group (UUID)	name, type (OFFICIEL/PERSONNALISE), is_public	id_anime → animes, id_creator → users
group_members	id_member (UUID)	is_moderator, joined_at	id_group → groups, id_user → users
group_messages	id_message (UUID)	content, created_at	id_group → groups, id_sender → users

3.4 Modèle de classes

Le modèle de classes représente les entités du domaine avec leurs attributs, méthodes et relations. Les classes principales sont User, UserAnime, Review, Follow, Conversation, Message, Group et GroupMessage.



Classe User

Attributs :

id (UUID), email (unique), password (hash bcrypt), pseudo (unique), bio (max 500 car.), role (USER | MODERATEUR | ADMIN), avatar (index couleur), isActive, createdAt

Méthodes :

register(), login() → JWT, logout() → blacklist, updateProfile(), changePassword(), follow(userId), unfollow(userId), getStatistics()

Classe Review

Attributs :

id, userId, animeId, rating (1-10), comment (max 2000 car.), visibility (PUBLIC | PRIVE), likesCount, createdAt, updatedAt

Contraintes :

Une seule review par (userId, animeId) — implémenté en UNIQUE CONSTRAINT SQL et en upsert côté service.

Classe Group

Attributs :

id, animeId, name, type (OFFICIEL | PERSONNALISE), isPublic, creatorId, memberCount, createdAt

Contraintes :

Les groupes OFFICIEL ne peuvent pas être supprimés — cette règle est appliquée au niveau du service, pas de la base de données.

3.5 Diagrammes UML

Diagramme de cas d'utilisation

Ce diagramme représente l'ensemble des fonctionnalités accessibles par chaque acteur. Le Visiteur (non connecté) peut uniquement rechercher des animés. L'Utilisateur connecté accède à toutes les fonctionnalités. La Jikan API est un système externe sollicité pour les données des animés.

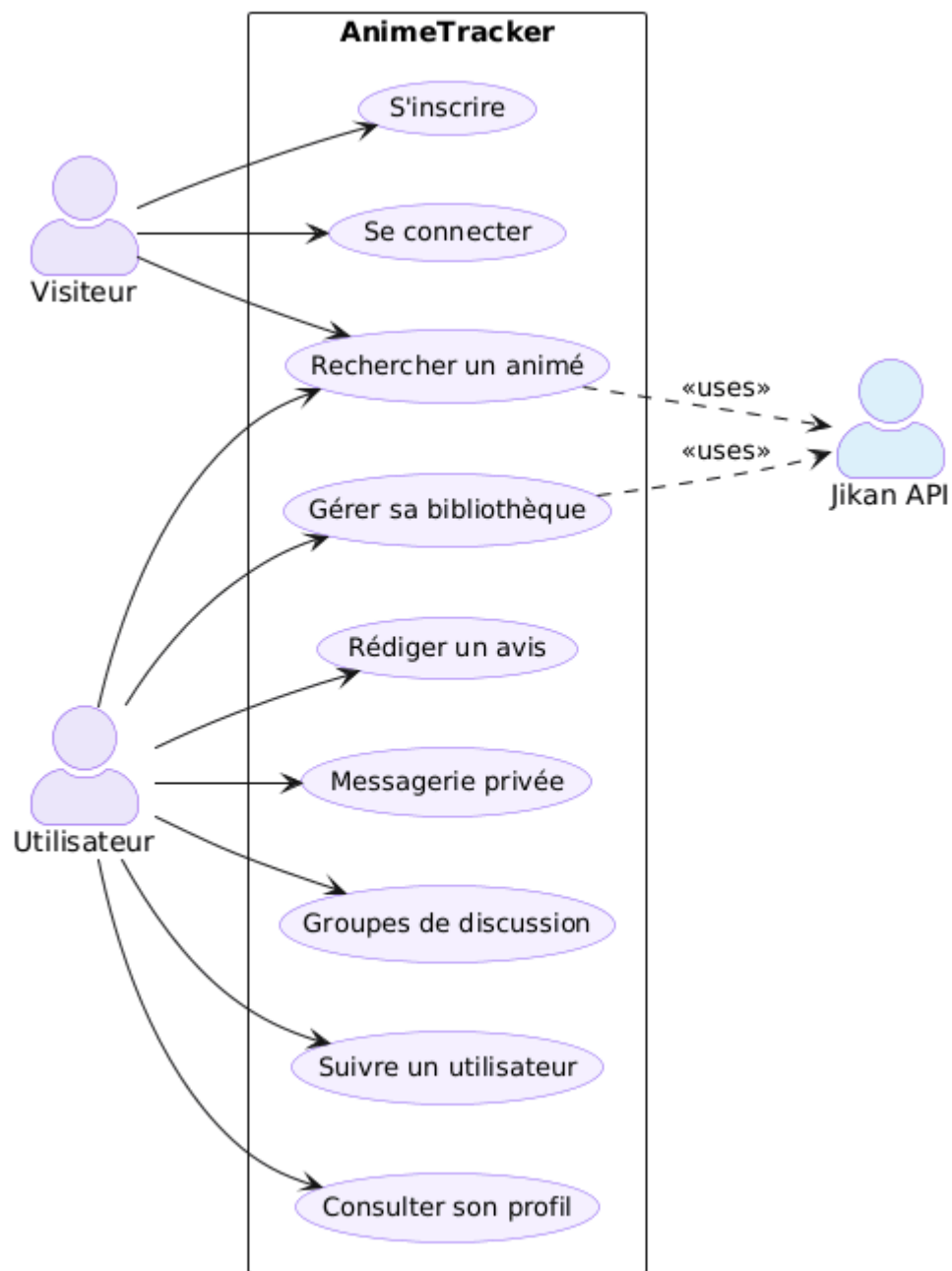


Diagramme de séquence — Connexion (login)

Flux complet d'une connexion : validation Zod, vérification bcrypt, génération du JWT et stockage sécurisé côté mobile.

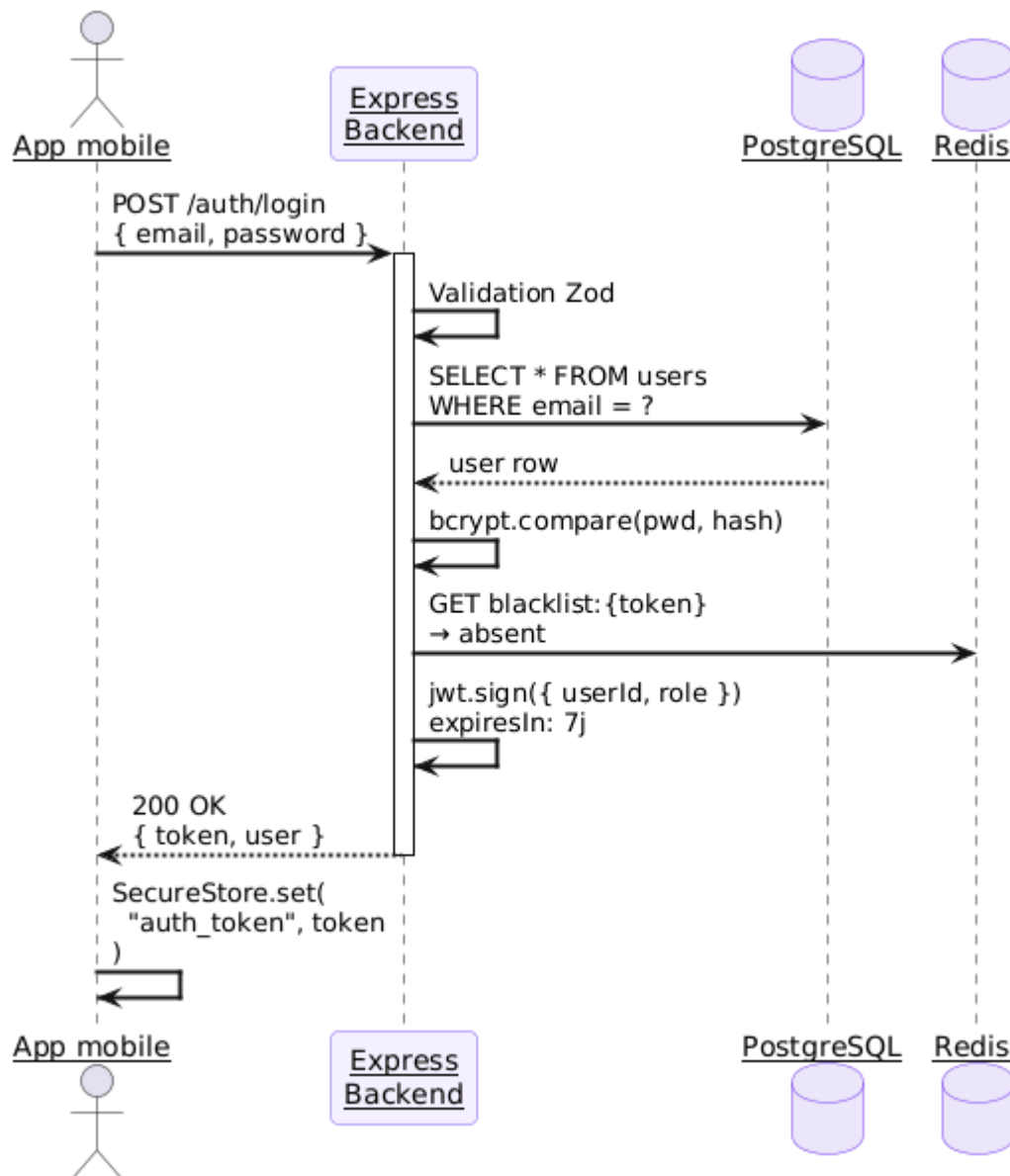


Diagramme de séquence — Ajout d'un animé à la bibliothèque

L'app récupère d'abord les données de l'animé depuis Jikan, puis envoie la demande d'ajout au backend qui effectue un UPSERT en base.

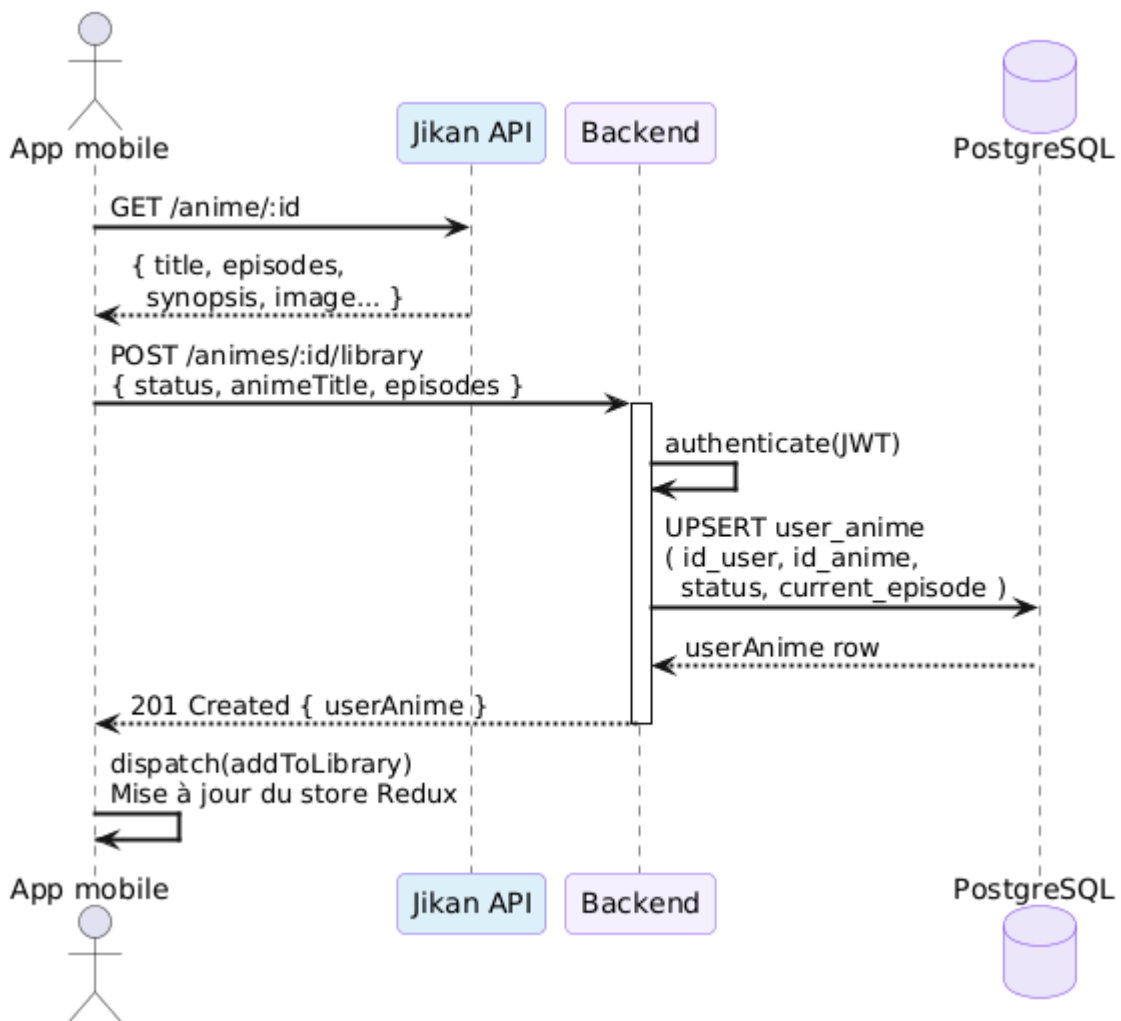
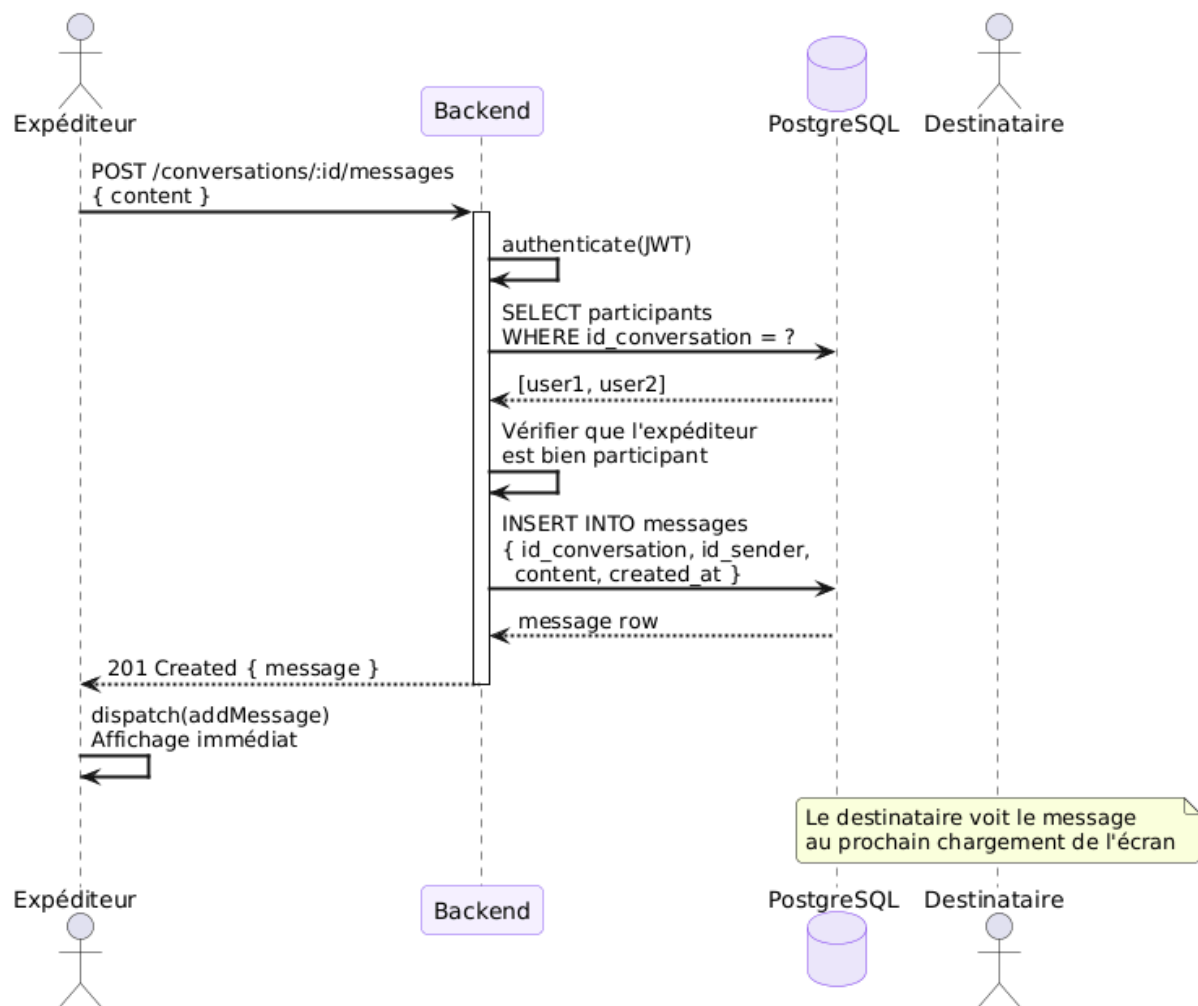


Diagramme de séquence — Envoi d'un message privé

Le backend vérifie que l'expéditeur est bien participant de la conversation avant d'insérer le message en base.



3.6 Maquettes et wireframes

Les maquettes ci-dessous représentent les 5 écrans principaux de l'application. Elles ont été produites pour valider la navigation, la hiérarchie visuelle et l'organisation des informations avant développement. La palette violette (#8B5CF6) et la barre de navigation en bas sont les éléments identitaires de l'interface.

9:41



AnimeTracker

Bienvenue !

Email

Mot de passe



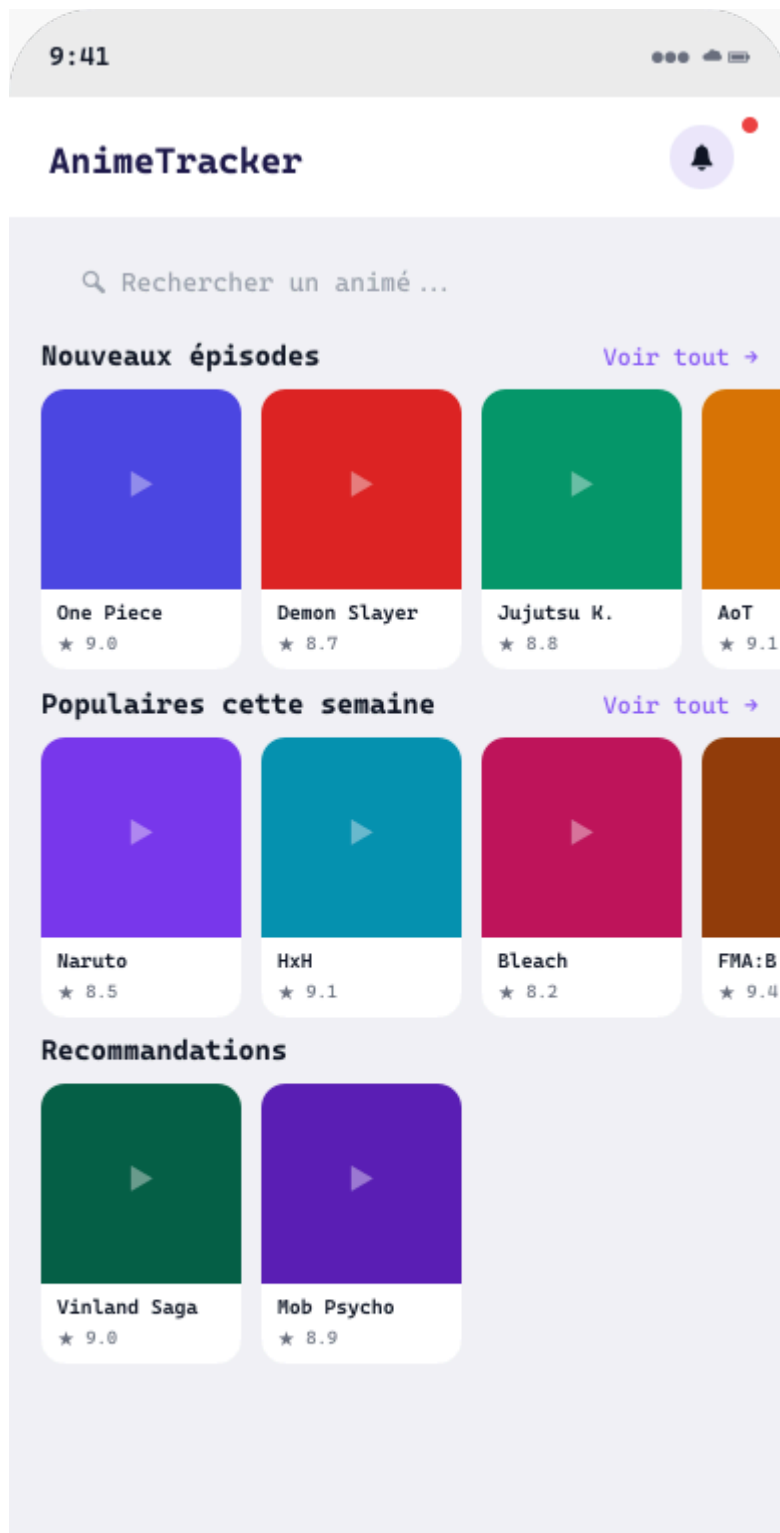
[Mot de passe oublié ?](#)

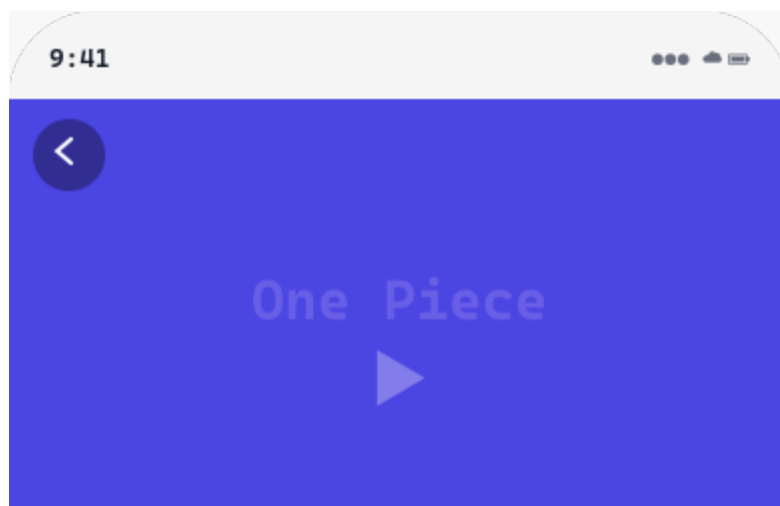
Se connecter

ou

Pas encore de compte ?

[S'inscrire](#)





One Piece

★ 9.0 / 10 (1 250 avis)

Shōnen

Adventure

Fantasy

1997 • 1100 épisodes • En cours

+ Ajouter à ma bibliothèque

Synopsis

Monkey D. Luffy rêve de devenir le roi des pirates en trouvant le trésor légendaire One Piece...

[Lire la suite](#)

Infos

Avis

Groupes

Studio	Toei Animation
Diffusion	Oct. 1999 – En cours
Source	Manga
Épisodes	1 100 (en cours)

9:41



Ma bibliothèque

En cours voir Terminés Abandonnés

En cours (4)

- **Attack on Titan**
En cours
24 / 25 épisodes
- **Demon Slayer**
En cours
18 / 26 épisodes
- **Jujutsu Kaisen**
En cours
23 / 24 épisodes
- **One Punch Man**
En cours
8 / 12 épisodes



Chat privé

Note : des captures d'écran réelles de l'application sur émulateur Android seront ajoutées en annexe pour illustrer le rendu final.

3.7 Charte graphique

Identité visuelle

L'identité visuelle d'AnimeTracker s'articule autour de la couleur violette (#8B5CF6), choisie pour son caractère moderne et son association avec l'univers créatif et la culture japonaise. Les mots-clés retenus : épuré, dynamique, accessible.

Palette de couleurs

Rôle	Couleur	Code hex	Usage
Primary	Violet	#8B5CF6	Boutons, liens, éléments actifs, badges "En cours"
Success	Vert	#10B981	Statut "Terminé", messages de succès
Warning	Orange	#F59E0B	Statut "À voir", alertes
Error	Rouge	#EF4444	Erreurs, suppression, statut "Abandonné"
Text primaire	Quasi-noir	#111827	Corps de texte principal
Text secondaire	Gris	#6B7280	Labels, métadonnées, timestamps
Background	Blanc/Gris clair	#F9FAFB	Fond des écrans et cartes

Le mode sombre est intégré nativement — chaque couleur a son équivalent dark (fond #0F172A, surface #1E293B), géré par un ThemeContext React qui persiste la préférence de l'utilisateur.

Typographie

Police principale : Inter (Google Fonts). Choisie pour son excellente lisibilité sur petits écrans et son support complet des caractères latin/accents.

Niveau	Taille	Poids	Usage
H1 — Titre de page	28px	Bold (700)	Titre des écrans principaux
H2 — Titre de section	24px	SemiBold (600)	Sections dans les écrans
Body Large	16px	Regular (400)	Texte principal, synopsis
Body Regular	14px	Regular (400)	Corps de texte standard
Caption	12px	Regular (400)	Timestamps, labels secondaires
Button	14-16px	SemiBold (600)	Libellés de boutons

Composants UI

- Boutons primaires : fond violet #8B5CF6, texte blanc, border-radius 8px, padding 12px 24px
- Cartes animés : fond blanc, bordure 1px #E5E7EB, border-radius 12px, ombre légère
- Bulles de chat : violettes (messages envoyés) / grises (messages reçus), border-radius asymétrique
- Badges de statut : fond coloré semi-transparent, texte coloré assorti, border-radius plein
- Champs de saisie : hauteur 48px, focus = bordure violette + halo semi-transparent
- Barre de navigation : 64px de hauteur, icône + label, actif en violet, inactif en gris

Partie 4 — Réalisation technique

Cette partie détaille les choix d'implémentation concrets : pourquoi chaque technologie a été retenue, comment le code est organisé, et des extraits commentés des points techniques les plus significatifs.

4.1 Justification des choix technologiques

React Native + Expo — frontend mobile

React Native permet d'écrire une seule base de code TypeScript compilée en composants natifs Android et iOS. Cela évite de maintenir deux projets séparés (Swift/Kotlin), divisant le temps de développement par deux pour un rendu vraiment natif — contrairement aux solutions WebView comme Cordova.

Expo ajoute une surcouche qui simplifie le démarrage : bundler préconfiguré, bibliothèques natives packagées, et l'app Expo Go pour tester sur téléphone physique via QR code sans passer par Android Studio.

Node.js + Express — backend API REST

Node.js partage le même langage que le frontend (TypeScript), ce qui élimine le changement de contexte mental. Son modèle event-loop non-bloquant est idéal pour une API à forte I/O (requêtes BDD, appels Jikan) : le serveur ne bloque pas en attendant une réponse, il traite d'autres requêtes en parallèle.

Express a été préféré à NestJS (trop verbeux pour ce projet) et Fastify (courbe d'apprentissage plus haute). Sa philosophie minimaliste laisse le développeur choisir comment organiser son code.

PostgreSQL — base de données relationnelle

Le schéma d'AnimeTracker est fortement relationnel : un utilisateur possède des animés, des avis, des messages, des abonnements. PostgreSQL est conçu pour ce type de données. MongoDB aurait mal convenu car les jointures sont nombreuses et la flexibilité des documents n'apporte aucune valeur ici.

PostgreSQL offre aussi des types ENUM natifs (statuts d'animé, visibilité des avis), des contraintes CHECK applicables au niveau base (ex. : `id_follower ≠ id_following` pour interdire de se suivre soi-même), et des UUID générés nativement.

Redis — cache pour la blacklist JWT

Redis est une base de données en mémoire (clé/valeur) utilisée exclusivement pour invalider les tokens JWT après déconnexion. À chaque requête authentifiée, le middleware vérifie si le token

figure dans Redis. Les opérations en mémoire sont 10 à 100 fois plus rapides qu'une requête SQL, ce qui est crucial pour cette vérification effectuée sur chaque appel API.

Docker — environnement de développement

Docker isole chaque service dans un conteneur indépendant. Un seul "docker compose up" démarre PostgreSQL, Redis et le backend dans les bonnes versions, dans le bon ordre. Cela garantit que l'environnement est identique sur tous les postes de développement.

4.2 Architecture backend

Structure des dossiers

backend/src/

- |— app.ts → Config Express, middlewares globaux, montage routes
- |— server.ts → Point d'entrée : connexion BDD + Redis puis HTTP
- |— config/
- | |— database.ts → Instance Sequelize, connexion PostgreSQL
- | |— redis.ts → Instance Redis
- |— models/ → Un fichier par table SQL
- |— routes/ → Endpoints HTTP, middlewares par route
- |— controllers/ → Extraction params, appel service, formatage réponse
- |— services/ → Logique métier pure, requêtes BDD
- |— middlewares/
- | |— authenticate.ts → Vérification JWT + blacklist Redis
- | |— errorHandler.ts → Handler d'erreur centralisé
- |— schemas/ → Schémas Zod (validation données entrantes)

Pattern Controllers / Services

Le controller gère uniquement HTTP : extraire les paramètres de la requête, appeler le service, formater la réponse. Le service contient la logique métier pure et ne connaît pas HTTP. Ce découpage rend les services testables indépendamment, sans simuler HTTP.

// Controller : gestion HTTP uniquement

```
export async function upsertReviewController(req, res, next) {  
  const animId = parseInt(req.params.animId, 10);  
  const review = await upsertReview(req.user.userId, animId, req.body);  
  res.status(201).json({ success: true, data: review });  
}
```

```
// Service : logique métier — ne connaît pas req/res
export async function upsertReview(userId, animeId, data) {
  const existing = await Review.findOne({
    where: { id_user: userId, id_anime: animeId }
  });
  if (existing) {
    await existing.update({ ...data, updated_at: new Date() });
    return existing;
  }
  return Review.create({ id_user: userId, id_anime: animeId, ...data });
}
```

Middleware d'authentification JWT

Le middleware authenticate est appliqué sur chaque route protégée. Il effectue trois vérifications dans l'ordre : présence du token, absence dans la blacklist Redis, validité de la signature JWT.

```
export async function authenticate(req, res, next) {
  const token = req.headers.authorization?.replace('Bearer ', '');
  if (!token) throw new AppError(401, 'Token manquant');
  // Vérification blacklist Redis (token invalidé après logout)
  const isBlacklisted = await redis.exists(`blacklist:${token}`);
  if (isBlacklisted) throw new AppError(401, 'Session expirée');
  // Vérification signature + expiration
  const payload = jwt.verify(token, process.env.JWT_SECRET);
  req.user = { userId: payload.userId, role: payload.role };
  next();
}
```

Validation des données avec Zod

Zod valide la forme des données reçues avant qu'elles n'atteignent la base. Si un client envoie rating: "abc", Zod renvoie une erreur 400 structurée directement affichable dans le frontend, sans que Sequelize n'ait à tenter l'insertion.

```
export const upsertReviewSchema = z.object({
  body: z.object({
    rating: z.number().min(0).max(10)
      .refine(v => (v * 2) % 1 === 0, 'Multiple de 0.5 requis'),
    comment: z.string().max(2000).optional(),
  })
});
```

```
visibility: z.enum(['PUBLIC', 'PRIVE']).optional(),
}),
});
```

4.2 bis Sécurité applicative — mesures de protection

La sécurité est traitée en couches successives : headers HTTP, limitation de débit, sanitisation des entrées, validation typée, authentification JWT et contrôle d'accès par rôle. Aucune de ces protections n'est suffisante seule — c'est leur combinaison qui garantit la robustesse.

1. En-têtes HTTP de sécurité — Helmet

Helmet configure automatiquement une série d'en-têtes HTTP qui protègent contre les attaques web classiques (XSS, clickjacking, MIME sniffing, injection de contenu). Chaque directive a un rôle précis :

// app.ts — configuration Helmet

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"], // seul le domaine courant peut charger des ressources
      scriptSrc: ["'self'"], // aucun script inline ou externe
      imgSrc: ["'self'", "data:", "https:"], // images depuis HTTPS uniquement
      objectSrc: ["'none'"], // interdit Flash et plugins
    },
  },
  frameguard: { action: "deny" }, // X-Frame-Options: DENY → anti-clickjacking
  noSniff: true, // X-Content-Type-Options: nosniff
  hsts: { maxAge: 31536000, includeSubDomains: true }, // force HTTPS 1 an
  hidePoweredBy: true, // masque X-Powered-By: Express
  xssFilter: true, // X-XSS-Protection: 1; mode=block
}));
```

2. Rate Limiting — protection contre la force brute

Un rate limiter global bloque les clients qui font trop de requêtes en peu de temps. C'est la première ligne de défense contre les attaques par force brute sur /auth/login et les tentatives d'énumération d'utilisateurs.

// app.ts — rate limiting global

```
app.use(rateLimit({
  windowMs: 15 * 60 * 1000, // fenêtre de 15 minutes
```

```

max: 100, // maximum 100 requêtes par IP par fenêtre
standardHeaders: true, // renvoie Retry-After dans les headers
message: {
  success: false,
  message: "Trop de requêtes, réessayez dans 15 minutes"
},
});

```

Effet : un attaquant qui tente de deviner un mot de passe sera bloqué après 100 essais sur 15 minutes. Avec un mot de passe de 8+ caractères (lettres + chiffres), la probabilité de succès par force brute est négligeable.

3. Sanitisation XSS — nettoyage des entrées

Le middleware `sanitizeBody` parcourt récursivement le body de chaque requête et échappe les caractères HTML dangereux (<, >, ", ') avant qu'ils n'atteignent la base de données. Cela complète la validation Zod en empêchant le stockage de contenu malveillant.

```

// middlewares/sanitize.ts

function sanitizeValue(value: unknown): unknown {
  if (typeof value === "string") {
    return value
      .replace(/&/g, "&amp;")
      .replace(/</g, "&lt;") // empêche <script>alert()</script>
      .replace(/>/g, "&gt;")
      .replace(/"/g, "&quot;")
      .replace(/'/g, "&#x27;");
  }
  if (typeof value === "object" && value !== null) {
    // Parcours récursif des objets et tableaux imbriqués
    return Object.fromEntries(
      Object.entries(value).map(([k, v]) => [k, sanitizeValue(v)])
    );
  }
  return value;
}

export const sanitizeBody = (req, _res, next) => {
  if (req.body) req.body = sanitizeValue(req.body);
  next();
}

```

```
};
```

4. Contrôle d'accès par rôle — RBAC avec requireRole()

Le middleware `requireRole()` implémente un contrôle d'accès basé sur les rôles (Role-Based Access Control). Il s'applique après `authenticate`, qui a déjà vérifié le JWT et peuplé `req.user.role`. Le panneau d'administration est entièrement protégé par cette combinaison.

```
// middlewares/authenticate.ts
export function requireRole(...roles: string[]) {
  return (req: AuthenticatedRequest, _res: Response, next: NextFunction) => {
    if (!req.user) {
      return next(new AppError(401, "Non authentifié"));
    }
    if (!roles.includes(req.user.role)) {
      return next(new AppError(403, "Accès refusé")); // 403 Forbidden
    }
    next();
  };
}
```

```
// routes/admin.ts — toutes les routes admin protégées en une ligne
router.use(authenticate, requireRole("ADMIN"));
router.get("/stats", asyncHandler(getStatsController));
router.get("/users", asyncHandler(listUsersController));
router.patch("/users/:id/role", asyncHandler(changeUserRoleController));
router.patch("/users/:id/suspend", asyncHandler(suspendUserController));
router.delete("/reviews/:reviewId", asyncHandler(deleteReviewController));
router.delete("/group-messages/:id", asyncHandler(deleteGroupMessageController));
```

Résultat : un utilisateur avec `role = "USER"` qui tente d'accéder à `/api/admin/stats` reçoit une réponse `403 Forbidden`. Le rôle est encodé dans le JWT signé côté serveur — il ne peut pas être falsifié côté client.

Synthèse des couches de sécurité

Chaque requête entrante traverse successivement :

- Helmet → headers HTTP défensifs (XSS, clickjacking, MIME sniffing)
- Rate Limiter → blocage si > 100 req / 15 min / IP
- Payload limit → rejet si body > 10 ko
- `sanitizeBody` → échappement HTML de toutes les chaînes

- Zod → validation typée et contraintes métier (format, longueur, enum)
- authenticate → vérification JWT + blacklist Redis
- requireRole → vérification du rôle (admin, modérateur...)
- Sequelize → requêtes paramétrées → immunité injection SQL
- UUID → clés primaires non-prédictibles → pas d'énumération

4.3 Architecture frontend

Structure des dossiers

```
src/
├── navigation/ → Routes typées, Auth Stack, Main Stack + Tab Bar
├── screens/
│   ├── auth/ → LoginScreen, RegisterScreen
│   └── main/ → HomeScreen, AnimeDetailScreen, ChatScreen...
├── store/
│   ├── index.ts → Configuration du store Redux
│   └── slices/ → Un slice par domaine métier
│       ├── authSlice.ts
│       ├── librarySlice.ts
│       ├── reviewsSlice.ts
│       ├── chatSlice.ts
│       ├── groupsSlice.ts
│       └── socialSlice.ts
├── services/
│   ├── apiClient.ts → Instance Axios avec intercepteurs JWT
│   └── jikanApi.ts → Appels API MyAnimeList
├── contexts/
│   └── ThemeContext.tsx → Thème clair/sombre persisté
└── types/index.ts → Interfaces TypeScript partagées
```

Gestion d'état — Redux Toolkit

Redux centralise l'état dans un store unique. Chaque slice gère un domaine métier avec ses thunks async (appels API) et ses reducers (comment le state change). Les sélecteurs permettent aux composants de lire l'état sans connaître sa structure interne.

// Thunk : appel API + mise à jour du store

```
export const createReview = createAsyncThunk(
```

```

'reviews/create',
async (reviewData, { rejectWithValue }) => {
  try {
    const { data } = await apiClient.post(
      `/animés/${reviewData.animéId}/reviews`,
      { rating: reviewData.rating, comment: reviewData.content,
        visibility: reviewData.isPublic ? 'PUBLIC' : 'PRIVE' }
    );
    return mapReview(data.data, reviewData); // BDD → format frontend
  } catch (err) {
    return rejectWithValue(err.message);
  }
};

// Reducer : upsert — remplace si déjà existant
.addCase(createReview.fulfilled, (state, action) => {
  const idx = state.reviews.findIndex(
    r => r.userId === action.payload.userId
    && r.animéId === action.payload.animéId
  );
  if (idx !== -1) state.reviews[idx] = action.payload;
  else state.reviews.push(action.payload);
})

```

Client HTTP — Axios avec intercepteurs

L'instance Axios est configurée une seule fois. Deux intercepteurs s'occupent de tout : l'un ajoute automatiquement le token JWT à chaque requête, l'autre normalise les erreurs en messages lisibles pour l'utilisateur.

```

const apiClient = axios.create({
  baseURL: process.env.EXPO_PUBLIC_API_URL, // http://10.0.2.2:3000/api
  timeout: 10000,
});

// Avant chaque requête : injecter le JWT
apiClient.interceptors.request.use(async (config) => {
  const token = await SecureStore.getItemAsync('auth_token');

```



```
if (token) config.headers.Authorization = `Bearer ${token}`;  
return config;  
});  
// Après chaque réponse : normaliser les erreurs  
apiClient.interceptors.response.use(  
  response => response,  
  (error) => {  
    const message = error.response?.data?.message  
    ?? 'Erreur réseau';  
    return Promise.reject(new Error(message));  
  }  
);
```

4.4 Base de données

Schéma des tables principales

Table	Colonnes clés	Contraintes notables
users	id_user (UUID PK), email, password, pseudo, role, avatar, is_active	UNIQUE(email), UNIQUE(pseudo)
user_anime	id_user_anime, id_user, id_anime, status, current_episode, is_favorite	UNIQUE(id_user, id_anime) — 1 entrée max par animé et par utilisateur
reviews	id_review, id_user, id_anime, rating, comment, visibility, likes_count	UNIQUE(id_user, id_anime) — 1 avis max par animé et par utilisateur
review_likes	id_like, id_review, id_user, created_at	UNIQUE(id_review, id_user) — 1 like max par avis et par utilisateur
follows	id_follow, id_follower, id_following, created_at	UNIQUE(follower, following), CHECK(follower ≠ following)
messages	id_message, id_conversation, id_sender, content, is_read, created_at	is_deleted pour suppression logique
groups	id_group, name, type, id_anime, id_creator	type ENUM: OFFICIEL ou PRIVE
group_messages	id_message, id_group, id_sender, content, created_at, deleted_at	deleted_at pour suppression logique (modération)

Associations Sequelize

Les associations sont définies dans un fichier centralisé associations.ts. Elles permettent à Sequelize de générer automatiquement les jointures SQL lors des requêtes avec include.

```
// models/associations.ts
User.hasMany(Review, { foreignKey: 'id_user', as: 'reviews' });
Review.belongsTo(User, { foreignKey: 'id_user', as: 'author' });
User.belongsToMany(User, {
  through: Follow,
  as: 'followers',
  foreignKey: 'id_following',
  otherKey: 'id_follower',
});
```

4.5 Fonctionnalités clés — points techniques

Authentification — flux complet

Le mot de passe est haché avec bcrypt (coût 10 = 1024 itérations) avant tout stockage. Le JWT est signé avec un secret d'environnement et expire après 7 jours. À la déconnexion, le token est placé en blacklist Redis avec une TTL égale à son temps de vie restant.

Bibliothèque personnelle — statuts et progression

Chaque entrée `user_anime` stocke le statut (`A_VOIR`, `EN_COURS`, `TERMINE`, `ABANDONNE`), l'épisode courant, le total d'épisodes et un booléen `is_favorite`. La contrainte `UNIQUE(id_user, id_anime)` est gérée à la fois en base et côté service (upsert : si l'entrée existe, on la met à jour).

Avis — visibilité publique/privée

Un avis est privé par défaut. L'endpoint `GET /animes/:id/reviews` ne retourne que les avis PUBLIC. Pour que l'auteur voie son propre avis privé après navigation, un endpoint dédié `GET /animes/:id/my-review` retourne l'avis de l'utilisateur connecté quelle que soit sa visibilité. Les deux sont appelés en parallèle au chargement de l'écran détail.

Groupe — persistance de l'appartenance

Redux ne persiste pas entre les sessions. L'appartenance à un groupe serait perdue au redémarrage. Solution : au montage d'`AnimeGroupScreen`, le thunk `checkAnimeGroup` appelle `GET /groups/anime/:animeId` qui retourne `{ group, memberCount, isMember }`. Si `isMember` est true, le groupe est ajouté à `joinedGroupIds` dans Redux — l'appartenance est restaurée depuis la BDD.

Système social — follow/unfollow

Le suivi d'un utilisateur implique trois couches coordonnées : le backend retourne les données complètes après un follow (pas juste 200 OK), le reducer Redux met à jour la liste des abonnés en mémoire immédiatement, et le sélecteur `selectIsFollowing` consulte `state.social.followers` (chargé au montage du profil consulté) plutôt que `state.social.following`.

4.6 Infrastructure Docker

docker-compose.yml (simplifié)

services:

postgres:

image: postgres:15

ports: ["5433:5432"] # 5433 hôte (5432 occupé par Postgres natif)

healthcheck:

test: ["CMD", "pg_isready"] # le backend attend ce signal

redis:

image: redis:7

ports: ["6380:6379"]

backend:

build: ./backend

ports: ["3000:3000"]

depends_on:

postgres: { condition: service_healthy }

redis: { condition: service_healthy }

Point d'attention rencontré : Docker Desktop sur Windows ne transmet pas les événements de modification de fichiers au système de fichiers Linux du conteneur. Nodemon ne détectait pas les changements. Solution : redémarrer manuellement le conteneur backend après chaque modification (docker restart animetracker_backend).

Partie 4 (suite) — Infrastructure et déploiement

Cette section documente l'infrastructure mise en place pour faire tourner AnimeTracker en local, les choix d'organisation des conteneurs Docker, la gestion des secrets et les perspectives de déploiement en production.

4.7 Architecture Docker — vue d'ensemble

L'ensemble de l'infrastructure backend est conteneurisée avec Docker. Trois conteneurs coopèrent via un réseau interne Docker, chacun isolé dans son propre environnement :

Conteneur	Image	Port hôte	Rôle
animetracker_postgres	postgres:15-alpine	5433 → 5432	Base de données relationnelle principale
animetracker_redis	redis:7-alpine	variable → 6379	Cache en mémoire — blacklist JWT
animetracker_backend	node:20-alpine (custom)	3000 → 3000	API REST Express + TypeScript

Le port PostgreSQL est remappé sur 5433 côté hôte (au lieu du standard 5432) car une installation native de PostgreSQL tournait déjà sur ce port sur la machine de développement. Ce genre de conflit de port est courant en développement local et Docker le résout proprement via le mapping de ports.

4.8 Fichier docker-compose.yml commenté

Le fichier docker-compose.yml décrit l'ensemble des services, leurs dépendances, leurs volumes et leurs variables d'environnement. Voici le fichier utilisé dans le projet, avec les points clés expliqués.

services:

```
# — PostgreSQL —————
```

```
postgres:
```

```
image: postgres:15-alpine # image légère (alpine)
```

```
container_name: animetracker_postgres
```

```
environment:
```

```
POSTGRES_DB: ${POSTGRES_DB}
```

```
POSTGRES_USER: ${POSTGRES_USER}
```

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}

ports:

- "5433:5432" # hôte:conteneur

volumes:

- postgres_data:/var/lib/postgresql/data # données persistées

- ./docker/postgres/init.sql:/docker-entrypoint-initdb.d/01_init.sql

healthcheck:

test: ["CMD-SHELL", "pg_isready -U \${POSTGRES_USER} -d \${POSTGRES_DB}"]

interval: 10s

retries: 5

— Redis

redis:

image: redis:7-alpine

container_name: animetracker_redis

ports:

- "\${REDIS_PORT}:6379"

command: redis-server --appendonly yes # persistance des données

healthcheck:

test: ["CMD", "redis-cli", "ping"]

— Backend Node.js

backend:

build:

context: ./backend

dockerfile: Dockerfile

depends_on:

postgres: { condition: service_healthy } # attend que PG soit prêt

redis: { condition: service_healthy } # attend que Redis soit prêt

environment:

POSTGRES_HOST: postgres # nom du service = hostname interne Docker

REDIS_HOST: redis

JWT_SECRET: \${JWT_SECRET}

volumes:

- ./backend:/app # montage du code source (hot-reload)

- /app/node_modules # node_modules du conteneur non écrasés

profiles:

- full # lancé seulement avec --profile full

volumes:

postgres_data: # volume nommé = données conservées entre redémarrages

redis_data:

Points clés du docker-compose.yml :

- depends_on avec condition: service_healthy — le backend ne démarre pas tant que PostgreSQL et Redis n'ont pas passé leur healthcheck. Sans ça, Node.js tenterait de se connecter avant que la base soit prête et crasherait.
- Volumes nommés (postgres_data, redis_data) — les données ne sont pas perdues à l'arrêt des conteneurs. Un docker compose down simple ne détruit pas les données ; seul un docker compose down -v le ferait.
- Montage ./backend:/app — le code source local est monté dans le conteneur. Toute modification de fichier est immédiatement visible dans le conteneur (hot-reload via ts-node-dev).
- Profile "full" sur le backend — permet de lancer uniquement la BDD et Redis (docker compose up) sans le backend conteneurisé, pour développer le backend en local (npm run dev) tout en ayant les services infra dans Docker.

4.9 Dockerfile du backend

Le Dockerfile du backend est volontairement minimaliste pour le développement. Il utilise Node.js 20 sur Alpine Linux (image légère, ~50 Mo) et lance le serveur en mode développement avec rechargement automatique.

```
FROM node:20-alpine
```

```
WORKDIR /app
```

```
# Copier les fichiers de dépendances en premier
```

```
# Docker met en cache cette couche — npm ci ne s'exécute que si
```

```
# package.json ou package-lock.json change
```

```
COPY package*.json ./
```

```
RUN npm ci --only=production=false
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["npm", "run", "dev"] # ts-node-dev avec --respawn
```

En production, ce Dockerfile serait en deux étapes (multi-stage build) : une étape de compilation TypeScript → JavaScript, puis une image finale sans les dépendances de développement ni le compilateur TypeScript, réduisant la taille de l'image finale de 400 Mo à ~80 Mo.

4.10 Gestion des variables d'environnement

Les variables d'environnement centralisent toutes les valeurs qui changent selon le contexte (développement, production) ou qui ne doivent pas être exposées dans le code source (mots de passe, clés secrètes).

Fichiers .env — organisation

Fichier	Commité dans git ?	Rôle
.env	Non ❌ (.gitignore)	Valeurs réelles utilisées en développement local
.env.dev	Oui ✅ (template)	Modèle avec valeurs par défaut — aucun secret réel

Le fichier .env.dev sert de documentation vivante : tout nouveau développeur sait exactement quelles variables configurer. Les valeurs de production (JWT_SECRET fort, mots de passe sécurisés) ne figurent que dans le .env local, jamais dans git.

Contenu du .env.dev (template)

```
# Frontend Expo
EXPO_PUBLIC_API_URL=http://localhost:3000/api
EXPO_PUBLIC_JIKAN_URL=https://api.jikan.moe/v4

# PostgreSQL
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=animetracker
POSTGRES_USER=animetracker_user
POSTGRES_PASSWORD=animetracker_dev_password

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379

# Backend
NODE_ENV=development
PORT=3000
JWT_SECRET=change_this_secret_in_production_min_32_chars
JWT_EXPIRES_IN=7d
JIKAN_API_URL=https://api.jikan.moe/v4
```

4.11 Initialisation automatique de la base de données

Le fichier `docker/postgres/init.sql` est monté dans le répertoire `/docker-entrpoint-initdb.d/` du conteneur PostgreSQL. Ce mécanisme natif de l'image officielle PostgreSQL exécute automatiquement tous les fichiers `.sql` présents dans ce dossier au premier démarrage du conteneur (quand le volume `postgres_data` est vide).

Le fichier `init.sql` crée dans l'ordre :

- L'extension `uuid-oss` — active la fonction `uuid_generate_v4()` pour générer des UUID automatiquement
- Les types `ENUM` — `user_role`, `anime_status`, `visibility_type`, `group_type`... Ces types contraignent les valeurs acceptables directement au niveau de la base de données
- Toutes les tables avec leurs colonnes, types, contraintes `PRIMARY KEY`, `UNIQUE`, `CHECK` et `FOREIGN KEY`
- Les index de performance sur les colonnes fréquemment filtrées (`id_user`, `id_anime`, `created_at`...)

-- Extrait de `init.sql`

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TYPE anime_status AS ENUM ('A_VOIR', 'EN_COURS', 'TERMINE', 'ABANDONNE');

CREATE TYPE visibility_type AS ENUM ('PUBLIC', 'PRIVE');

CREATE TABLE "user" (
  id_user UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(255) UNIQUE NOT NULL,
  pseudo VARCHAR(50) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  role user_role DEFAULT 'USER' NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
  CONSTRAINT chk_email_format CHECK (email ~* '^[A-Za-z0-9._%+-]+@...$'),
  CONSTRAINT chk_pseudo_length CHECK (LENGTH(pseudo) >= 3)
);
```

4.12 Procédure de démarrage du projet

Voici les étapes complètes pour démarrer l'environnement de développement depuis zéro sur une nouvelle machine.

Prérequis

- Docker Desktop installé et démarré
- Node.js 20+ installé
- Expo Go installé sur le téléphone (ou émulateur Android configuré)

Étapes

1. Cloner le dépôt et configurer l'environnement

```
git clone <repo>
cp .env.dev .env # copier le template
# éditer .env avec les vraies valeurs si nécessaire
```

2. Démarrer PostgreSQL et Redis

```
docker compose up -d
# PostgreSQL s'initialise automatiquement (init.sql)
# Attendre que les healthchecks passent (~15s)
```

3. Démarrer le backend

```
cd backend
npm install
npm run dev # ts-node-dev, port 3000
```

4. Démarrer le frontend Expo

```
cd .. # racine du projet
npm install
npx expo start
# Scanner le QR code avec Expo Go
# ou appuyer sur "a" pour l'émulateur Android
```

4.13 Contraintes liées à Windows et Docker Desktop

Le développement sur Windows avec Docker Desktop soulève une contrainte spécifique importante : Docker Desktop sur Windows exécute les conteneurs dans une machine virtuelle Linux (WSL2). Le système de fichiers Windows n'émet pas d'événements inotify nativement, ce qui empêche les outils de rechargement automatique (comme ts-node-dev ou nodemon) de détecter les modifications de fichiers depuis l'intérieur du conteneur.

Symptôme	Cause	Solution adoptée
ts-node-dev ne redémarre pas après modification d'un fichier	Pas d'événements inotify entre Windows NTFS et le conteneur Linux	Backend lancé en local (npm run dev) plutôt qu'en conteneur pendant le développement
docker compose up --profile full relance le backend en conteneur	Volume monté depuis Windows → pas de hot-reload fiable	Redémarrage manuel : docker restart animetracker_backend

La solution pragmatique retenue : PostgreSQL et Redis tournent dans Docker (services stables qui ne changent pas), tandis que le backend Node.js tourne directement sur la machine hôte Windows pendant le développement actif. Le docker-compose.yml propose le profil "full" pour les rares cas où l'on veut tout conteneuriser (par exemple pour simuler l'environnement de production).

4.14 Perspectives de déploiement en production

Le projet est en environnement de développement local. Un déploiement en production nécessiterait les adaptations suivantes :

Élément	Développement (actuel)	Production (cible)
Backend	ts-node-dev (rechargement auto, lent)	Build TypeScript → JS, image Docker optimisée (multi-stage)
Base de données	Docker local, données sur le disque hôte	Service managé (ex. Supabase, RDS) avec sauvegardes automatiques
Secrets	.env local non commité	Variables d'environnement injectées par la plateforme (Railway, Render, VPS)
URL de l'API	http://10.0.2.2:3000 (émulateur)	https://api.mondomaine.com (HTTPS obligatoire)
CORS	Ouvert ou wildcard	Restreint aux origines Expo/app store uniquement
Logs	Console Node.js	Service de logs centralisé (ex. Logtail, Sentry)
Frontend mobile	Expo Go (développement)	Build EAS (Expo Application Services) → APK/AAB Google Play

La conteneurisation avec Docker simplifie ce passage en production : l'image construite en local est identique à celle qui tournerait sur un VPS ou une plateforme cloud. Il suffit de changer les variables d'environnement et de pointer vers la bonne base de données.

Partie 5 — Tests et qualité

La stratégie de tests d'AnimeTracker combine des tests automatisés (Jest) pour la logique métier et les routes HTTP, et des tests manuels sur émulateur Android pour valider le comportement réel de l'application.

5.1 Stratégie de tests

Niveau	Outil	Objectif
Tests unitaires	Jest + mocks	Valider la logique métier de chaque service isolément, sans BDD réelle
Tests d'intégration	Jest + Supertest	Valider les routes HTTP complètes (requête → réponse) avec middlewares
Tests manuels	Émulateur Android (API 33)	Valider le comportement réel de l'application sur toutes les fonctionnalités

Environnement de test :

- Émulateur Android — Android Studio, API level 33
- Backend Node.js/Express en local — port 3000
- PostgreSQL dans Docker — port 5433
- Redis dans Docker — port 6380
- Tous les modèles Sequelize, la BDD et Redis sont mockés pendant les tests Jest — aucune connexion réelle

5.2 Tests automatisés Jest — résultats

authService.test.ts — 15 tests unitaires

#	Cas de test	Résultat
1	Inscription avec données valides → Token JWT + objet user	✓ PASS
2	Inscription avec email déjà existant → AppError 409	✓ PASS
3	Inscription avec pseudo déjà pris → AppError 409	✓ PASS
4	Connexion avec identifiants corrects → Token JWT	✓ PASS
5	Connexion avec email inexistant → AppError 401	✓ PASS
6	Connexion avec mauvais mot de passe → AppError 401	✓ PASS
7	Déconnexion → Token ajouté dans Redis avec TTL	✓ PASS
8	Vérification token blacklisté → AppError 401	✓ PASS
9	Vérification token valide → Payload JWT retourné	✓ PASS
10	Vérification token malformé → AppError 401	✓ PASS
11	Mot de passe haché avec bcrypt → hash ≠ mot de passe clair	✓ PASS
12	Comparaison bcrypt — bon mot de passe → true	✓ PASS
13	Comparaison bcrypt — mauvais mot de passe → false	✓ PASS
14	Token JWT expiré → AppError 401	✓ PASS
15	Payload JWT contient userId et role	✓ PASS

userService.test.ts — 12 tests unitaires

#	Cas de test	Résultat
1	Récupérer profil existant → objet user sans champ password	✓ PASS
2	Récupérer profil inexistant → AppError 404	✓ PASS
3	Mettre à jour pseudo valide → mis à jour en BDD	✓ PASS
4	Mettre à jour avec pseudo déjà pris → AppError 409	✓ PASS
5	Changer mot de passe — ancien correct → nouveau hash sauvegardé	✓ PASS
6	Changer mot de passe — ancien incorrect → AppError 401	✓ PASS
7	Suivre un utilisateur → relation créée en BDD	✓ PASS
8	Suivre un utilisateur déjà suivi → AppError 409	✓ PASS
9	Se suivre soi-même → AppError 400	✓ PASS
10	Se désabonner d'un utilisateur suivi → relation supprimée	✓ PASS
11	Se désabonner d'un non-suivi → AppError 404	✓ PASS
12	Rechercher des utilisateurs → liste filtrée retournée	✓ PASS

auth.test.ts — 11 tests d'intégration

#	Route	Cas de test	Code HTTP	Résultat
1	POST /auth/register	Inscription valide	201	✓ PASS
2	POST /auth/register	Email manquant	400	✓ PASS
3	POST /auth/register	Mot de passe trop court	400	✓ PASS
4	POST /auth/register	Email déjà utilisé	409	✓ PASS
5	POST /auth/login	Connexion valide	200	✓ PASS
6	POST /auth/login	Mauvais mot de passe	401	✓ PASS
7	POST /auth/login	Utilisateur inexistant	401	✓ PASS
8	POST /auth/logout	Déconnexion avec token valide	200	✓ PASS
9	POST /auth/logout	Sans token	401	✓ PASS
10	GET /api/health	Endpoint de santé	200	✓ PASS
11	(route inconnue)	404 Not Found	404	✓ PASS

Résumé Jest

Test Suites : 3 passed, 3 total

Tests : 38 passed, 38 total (0 failed)

Snapshots : 0

Durée : ~4 secondes

5.3 Tests manuels — émulateur Android

Domaine	Tests	Passés	Taux
Authentification (M-01 à M-07)	7	7	100%
Bibliothèque personnelle (M-08 à M-15)	8	8	100%
Avis — Reviews (M-16 à M-22)	7	7	100%
Messagerie privée (M-23 à M-28)	6	6	100%
Groupes de discussion (M-29 à M-32)	4	4	100%
Social — abonnements (M-33 à M-37)	5	5	100%
Profil utilisateur (M-38 à M-41)	4	4	100%
Total	41	41	100%

5.4 Couverture fonctionnelle

Fonctionnalité	Tests automatisés	Tests manuels	Couverture
Inscription / Connexion	✓ Oui	✓ Oui	Complète
Gestion de profil	✓ Partiel (service)	✓ Oui	Complète
Bibliothèque animés	✗ Non	✓ Oui	Manuelle
Avis (reviews)	✗ Non	✓ Oui	Manuelle
Messagerie privée	✗ Non	✓ Oui	Manuelle
Groupes de discussion	✗ Non	✓ Oui	Manuelle
Social (follow)	✓ Partiel (service)	✓ Oui	Complète
Sécurité JWT	✓ Oui	✓ Oui	Complète
Validation Zod	✓ Oui	✓ Oui	Complète

Les tests automatisés couvrent principalement la couche service (logique métier) et les routes d'authentification. Les fonctionnalités métier complexes (bibliothèque, messagerie, groupes) sont validées par tests manuels sur émulateur Android. C'est un axe d'amélioration identifié pour une version future.

Partie 6 — Bilan et rétrospective

Ce bilan revient honnêtement sur ce qui a bien fonctionné, les difficultés rencontrées et la façon dont elles ont été résolues, et ce qui pourrait être amélioré dans une prochaine version.

6.1 Ce qui a bien fonctionné

Architecture découplée front/back

La séparation claire entre l'application mobile et l'API REST a permis de faire évoluer les deux côtés indépendamment. Un changement de schéma en base n'impliquait de modifier que le service backend et le slice Redux concerné — les écrans n'étaient pas impactés.

Redux Toolkit pour la gestion d'état

Le découpage en slices par domaine métier (auth, library, reviews, social, groups, chat) a rendu la base de code claire et maintenable. Les AsyncThunk standardisent tous les appels API avec leurs états pending/fulfilled/rejected.

Docker pour l'environnement de développement

Dockeriser PostgreSQL et Redis a produit un environnement reproductible en une commande. Cela a évité les problèmes d'installation locale et permis de travailler avec des versions précises des services.

TypeScript full-stack

Avoir TypeScript sur les deux couches a permis de détecter en amont des erreurs de mapping (snake_case BDD vs camelCase frontend) qui auraient été silencieuses en JavaScript. Les interfaces partagées entre services et slices servent de contrat.

6.2 Difficultés rencontrées et solutions apportées

Difficulté	Cause	Solution apportée
Persistance de l'appartenance aux groupes	Redux ne persiste pas entre sessions — au redémarrage, <code>joinedGroupIds</code> était vide	Endpoint <code>GET /groups/anime/:animeId</code> retournant <code>isMember</code> . Appelé au montage d' <code>AnimeGroupScreen</code> pour restaurer l'état depuis la BDD.
Avis privé qui disparaît après navigation	<code>GET /animes/:id/reviews</code> ne retourne que les avis PUBLIC par conception	Ajout de <code>GET /animes/:id/my-review</code> (authentifié) retournant l'avis de l'utilisateur quelle que soit sa visibilité. Appelé en parallèle au chargement.
Bouton Follow/Unfollow toujours désactivé	<code>selectIsFollowing</code> consultait <code>state.social.following</code> (les gens que l'autre suit) au lieu de <code>state.social.followers</code>	Correction du sélecteur + le backend retourne désormais les données complètes après un follow pour mettre à jour le store immédiatement.
Tests Jest qui échouent après ajout de modèles	Les nouveaux modèles (Conversation, Group...) initialisaient Sequelize à l'import, mais la BDD était mockée	Ajout de <code>jest.mock()</code> pour chaque nouveau modèle dans le fichier de setup des tests. Pratique documentée pour les futures ajouts.
Communication émulateur Android ↔ backend	localhost ne pointe pas vers la machine hôte depuis l'émulateur Android	URL de base configurée à <code>http://10.0.2.2:3000</code> (alias Android pour 127.0.0.1 de l'hôte). Documenté dans la config.

6.3 Axes d'amélioration

- Tests automatisés des fonctionnalités métier — bibliothèque, messagerie et groupes sont uniquement couverts par des tests manuels. Des tests d'intégration supplémentaires augmenteraient la fiabilité.
- Refresh token JWT — la gestion actuelle force une déconnexion à l'expiration. Un mécanisme de refresh token éviterait les interruptions inattendues.
- Messagerie en temps réel — le chat est actuellement en mode "pull". Socket.IO permettrait des mises à jour instantanées avec indicateur de frappe.
- Pagination des listes — les endpoints retournent toutes les données. Un système `limit/offset` est nécessaire pour les utilisateurs avec de nombreux animés ou messages.
- Upload d'avatar — le profil utilise un index de couleur prédéfini. Un vrai upload vers S3 ou Cloudinary offrirait des avatars personnalisés.
- Tests sur device physique et iOS — les tests manuels ont été réalisés sur émulateur Android uniquement.

6.4 Compétences acquises

Architecture full-stack mobile

Concevoir une application de bout en bout — de la base de données relationnelle jusqu'à l'interface utilisateur mobile — en maintenant la cohérence entre les couches (typage partagé, conventions de nommage, gestion des erreurs) est le défi principal de ce projet. Il a été résolu.

Gestion d'état complexe

Gérer l'état d'une application avec de multiples domaines interdépendants (auth, social, reviews...) a nécessité une compréhension approfondie de Redux : sélecteurs, thunks, patterns d'optimisation pour éviter les re-renders inutiles.

Sécurité applicative

L'implémentation de l'authentification JWT, du rate limiting, de la validation Zod et de la blacklist Redis a concrétisé les concepts de sécurité web dans un contexte réel. La compréhension des vecteurs d'attaque (injection, force brute, token replay) guide chaque choix de conception.

Debugging méthodique

Les bugs complexes (social, persistance des groupes) ont nécessité une approche par hypothèses successives : isoler, reproduire, identifier la cause racine, corriger, vérifier. Cette méthode est applicable à tout projet.

6.5 Conclusion

AnimeTracker a été développé avec succès en respectant le périmètre fonctionnel défini. Les 7 fonctionnalités principales sont opérationnelles et testées. Les 38 tests automatisés passent à 100%, et les 41 tests manuels sur émulateur Android valident le comportement réel de l'application.

Les difficultés rencontrées — persistance d'état, gestion de la visibilité des données, conventions de mapping — sont représentatives des défis réels du développement full-stack mobile. Chaque problème a été résolu par une approche méthodique, souvent en ajoutant une couche d'API dédiée plutôt qu'en contournant les contraintes côté frontend.

Ce projet valide la capacité à concevoir, développer et tester une application mobile professionnelle de bout en bout, en maîtrisant l'ensemble de la chaîne technologique — de la base de données PostgreSQL jusqu'à l'interface React Native sur Android.

AnimeTracker — Dossier CDA — Mai 2026

Annexe A — Liste complète des endpoints API

L'API REST d'AnimeTracker est accessible sous le préfixe `/api`. Toutes les routes marquées  nécessitent un token JWT valide dans l'en-tête `Authorization: Bearer <token>`.

Authentification — `/api/auth`

Méthode	Route	Auth	Description	Code
GET	<code>/api/health</code>		Vérification que le serveur répond	200
POST	<code>/api/auth/register</code>		Inscription — { email, password, pseudo }	201
POST	<code>/api/auth/login</code>		Connexion — { email, password } → JWT	200
POST	<code>/api/auth/logout</code>		Déconnexion — invalide le token dans Redis	200
GET	<code>/api/auth/me</code>		Récupérer son propre profil depuis le token	200

Utilisateurs — `/api/users`

Méthode	Route	Auth	Description	Code
GET	/api/users/search?q=		Rechercher des utilisateurs par pseudo	200
GET	/api/users/me		Récupérer son profil complet	200
PATCH	/api/users/me		Modifier pseudo, bio, avatar	200
PATCH	/api/users/me/password		Changer son mot de passe	200
DELETE	/api/users/me		Supprimer son compte (RGPD)	200
GET	/api/users/me/data		Exporter ses données personnelles (RGPD)	200
GET	/api/users/me/animes		Récupérer sa bibliothèque complète	200
POST	/api/users/me/animes		Ajouter un animé à sa bibliothèque	201
PATCH	/api/users/me/animes/:animeId		Modifier statut, épisode, favori	200
DELETE	/api/users/me/animes/:animeId		Retirer un animé de sa bibliothèque	200
GET	/api/users/me/stats		Statistiques de visionnage	200
DELETE	/api/users/me/reviews/:id		Supprimer un de ses avis	200
POST	/api/users/me/reviews/:id/like		Liker / unliker un avis public	200
GET	/api/users/:userId		Profil public d'un utilisateur	200
GET	/api/users/:userId/followers		Liste des abonnés d'un utilisateur	200
GET	/api/users/:userId/following		Liste des abonnements d'un utilisateur	200
POST	/api/users/:userId/follow		S'abonner à un utilisateur	201
DELETE	/api/users/:userId/follow		Se désabonner d'un utilisateur	200

Animés — /api/animes

Méthode	Route	Auth	Description	Code
GET	/api/animes/search?q=		Rechercher des animés (via Jikan API)	200
GET	/api/animes/top		Top animés par score MyAnimeList	200
GET	/api/animes/:id		Détail d'un animé par son ID Jikan	200
GET	/api/animes/:id/reviews		Avis publics d'un animé (triés par likes)	200
GET	/api/animes/:id/my-review		Récupérer son propre avis (public ou privé)	200
POST	/api/animes/:id/reviews		Créer ou mettre à jour son avis (upsert)	201
DELETE	/api/animes/:id/reviews		Supprimer son avis sur un animé	200

Messagerie privée — /api/conversations

Méthode	Route	Auth	Description	Code
GET	/api/conversations		Liste de toutes ses conversations	200
POST	/api/conversations/with/:recipientId		Créer ou récupérer une conversation avec un utilisateur	200
GET	/api/conversations/:id/messages		Historique des messages d'une conversation	200
POST	/api/conversations/:id/messages		Envoyer un message — { content }	201
PATCH	/api/conversations/:id/read		Marquer les messages comme lus	200
DELETE	/api/conversations/:id		Supprimer une conversation	200

Groupes de discussion — /api/groups

Méthode	Route	Auth	Description	Code
GET	/api/groups/anime/:animeld		Statut du groupe officiel d'un animé + isMember	200
POST	/api/groups/anime/:animeld		Rejoindre (ou créer) le groupe officiel d'un animé	200
GET	/api/groups/:groupid		Informations d'un groupe	200
POST	/api/groups/:groupid/join		Rejoindre un groupe	200
DELETE	/api/groups/:groupid/leave		Quitter un groupe	200
GET	/api/groups/:groupid/messages		Messages d'un groupe de discussion	200
POST	/api/groups/:groupid/messages		Envoyer un message dans un groupe — { content }	201

Récapitulatif

Domaine	Nb de routes	Routes publiques	Routes authentifiées
Authentification	5	2	3
Utilisateurs	18	3	15
Animés	7	3	4
Messagerie	6	0	6
Groupes	7	0	7
Total	43	8	35

Annexe B — Captures d'écran de l'application

Captures d'écran réelles de l'application AnimeTracker sur émulateur Android (Pixel 10 Pro XL, API level 33). Les données affichées sont des données de test.

Accueil et Recherche

9:38



Bonjour, TestUser !



Découvrez les meilleurs animes



★ 9.3

Frieren: Beyond Journey's End
TV



★ 9.1

Steel Ball Run: JoJo's Bizarre
Adventure
ONA



★ 9.1

Fullmetal Alchemist: Brotherhood
TV



★ 9.1

Chainsaw Man - The Movie: Reze
Arc
Movie



★ 9.1



★ 9.1



Accueil



Recherche



Bibliothèque



Messages



Profil



Recherche

🔍 frieren



Beyond Journey's End



Frieren: Beyond Journey's End
Season 2

TV



Frieren: Beyond Journey's End -
Golden Land Arc

TV



Frieren: Beyond Journey's End -
Golden Land Arc

TV



Accueil



Recherche



Bibliothèque



Messages



Profil

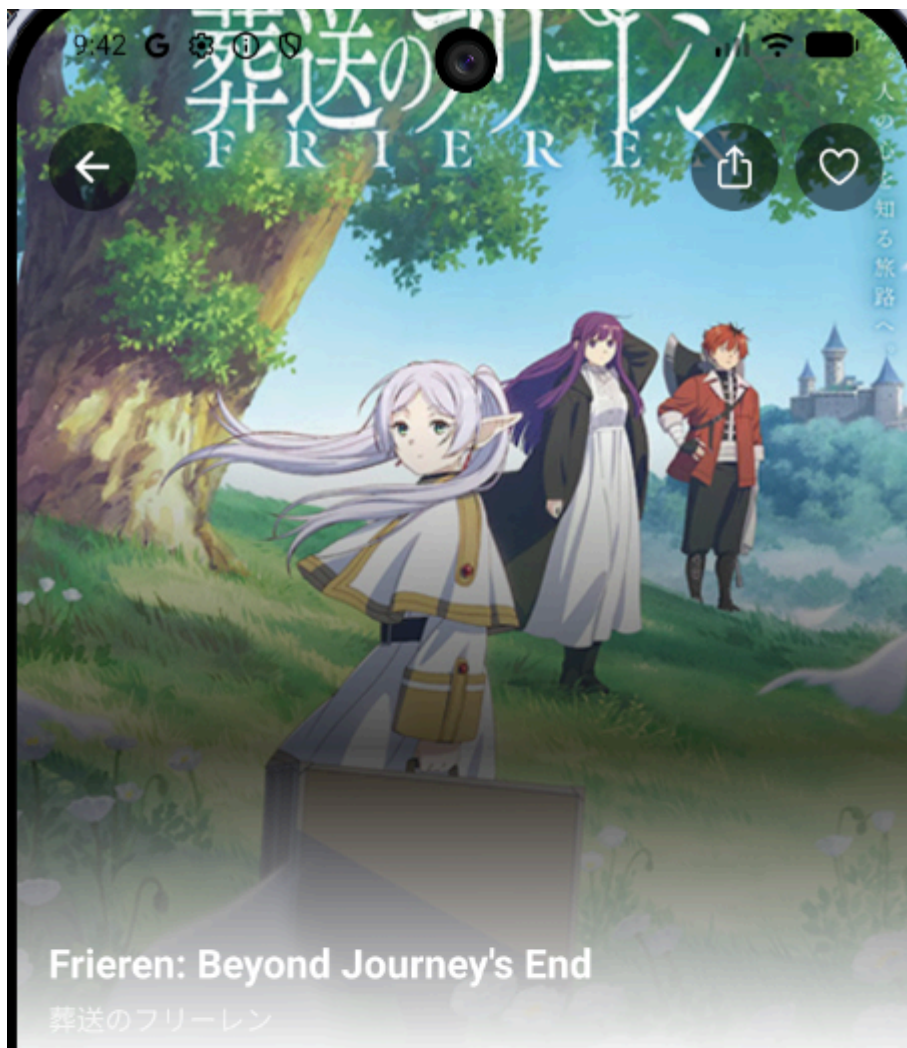
Fiche animé

9:42

葬送のフリーレン

FRIEREN

人々の知る旅路へ



Frieren: Beyond Journey's End

葬送のフリーレン

★ MAL

9.3

♥ Ma note

-

TV

28 épisodes

Finished Airing

Adventure

Award Winning

Drama

Fantasy

Terminé

Groupe de discussion

Discutez avec d'autres fans

>

Synopsis

During their decade-long quest to defeat the Demon King, the members of the hero's party—Himmel himself, the priest Heiter, the dwarf warrior Eisen, and the elven mage Frieren—forge bonds through adventures and battles, creating unforgettable precious ...



9:41



Terminé

**Groupe de discussion**

Discutez avec d'autres fans

**Synopsis**

During their decade-long quest to defeat the Demon King, the members of the hero's party—Himmel himself, the priest Heiter, the dwarf warrior Eisen, and the elven mage Frieren—forge bonds through adventures and battles, creating unforgettable precious ...

[Voir plus](#)**Informations**

Studio

Madhouse



Diffusion

Sep 29, 2023 to Mar 22, 2024



Source

Manga



Durée

24 min per ep



Classification

PG-13 - Teens 13 or older

Note privée[Ajouter une note privée](#)**Mon avis**[Écrire mon avis](#)**Avis de la communauté (1)**

pouet

★ 10.0

WOW INCROYABLE



1

28/05/2026



Fiche animé — cover, note MAL, genres, statut Synopsis, infos studio, avis de la communauté

Bibliothèque et Messagerie

Ma Bibliothèque

2 animes au total



Tout

2



Favoris

0



En cours

1



À voir

0



Terminé

1



Hunter x Hunter

En cours

Épisode 56/148



Accueil



Recherche



Bibliothèque



Messages



Profil



9:44



Messages



pouet
Salut

Hier



Accueil



Recherche



Bibliothèque



Messages



Profil

Profil utilisateur

Profil



TestUser

test@test.fr

Ma bibliothèque



2

Total



1

En cours



1

Terminés



0

À voir

Visionnage



56

Épisodes vus



22h 24min

Temps total

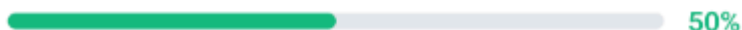


-

Note moyenne

Progression

Taux de complétion



50%



Accueil



Recherche



Bibliothèque



Messages



Profil



9:45



1

Terminés



0

À voir

Visionnage



56

Épisodes vus



22h 24min

Temps total

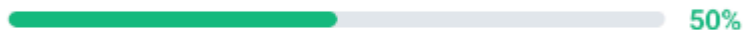


-

Note moyenne

Progression

Taux de complétion



0

Favoris



2

Cette semaine



0

Abandonnés

Social



1

Abonnés



1

Abonnements



0

Mes avis

 [Se déconnecter](#)

AnimeTracker v1.0.0



Accueil



Recherche



Bibliothèque



Messagerie

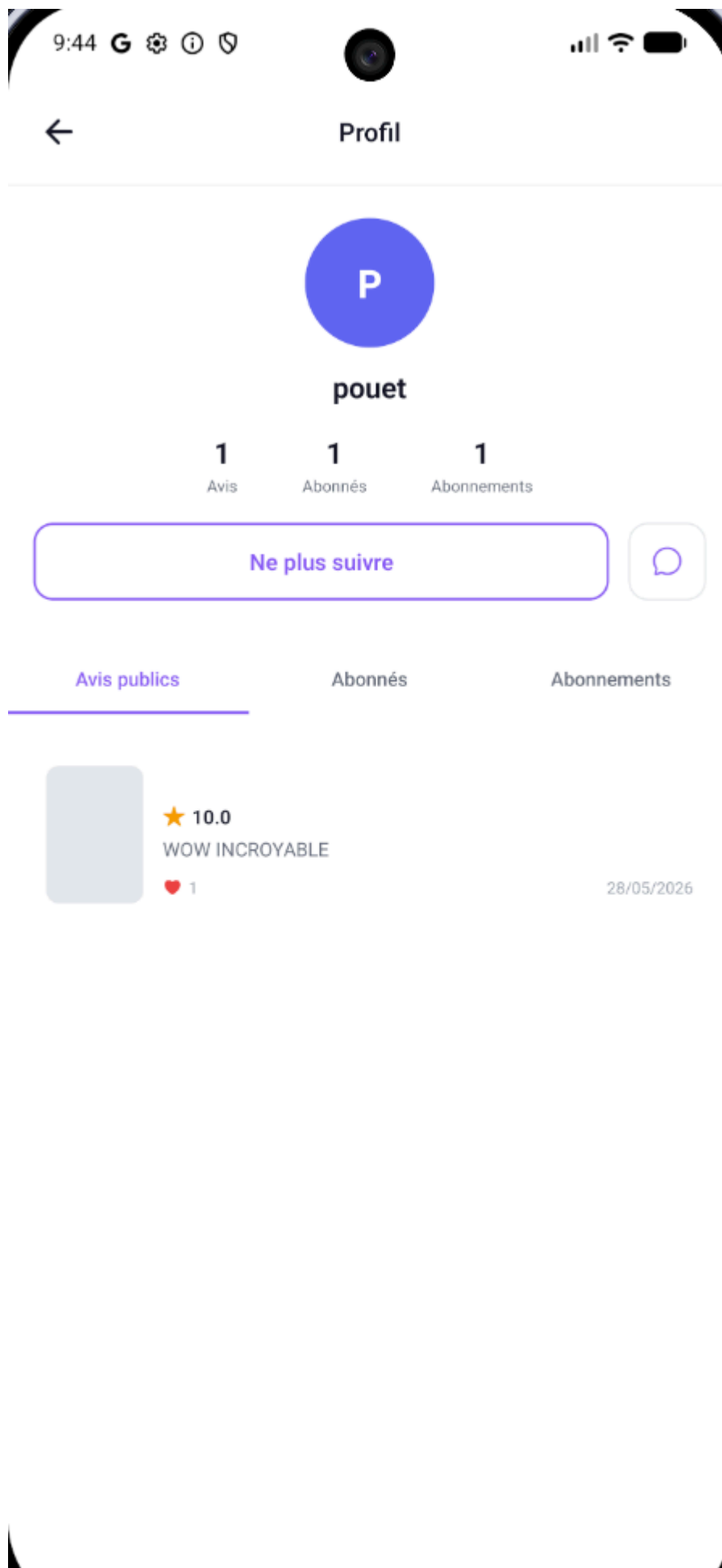


Profil



Profil — statistiques de bibliothèque Visionnage, progression et données sociales

Profil public





Profil public d'un autre utilisateur — avis publics, abonnés, bouton S'abonner/Ne plus suivre